

8/3/2026

TP1

Projeto de Bloco: Ciência da Computação

Professor(a): Ricardo Pires Mesquita

Estou utilizando o WSL

Parte 1

Inicialmente executei o código abaixo de curiosidade para verificar quantos linhas(arquivos) seriam adicionados no arquivo texto gerado para realizar a tarefa solicitada desta parte 1 e redireciono os para /dev/null(buraco negro do linux).

O retorno foi que seriam encontrados cerca de 30979 arquivos, mais de 30 mil.

```
Ubuntu
samuel@SamuelH:/$ sudo find . -path ./mnt -prune -o -type f -print 2>/dev/null | wc -l
30979
samuel@SamuelH:/$
```

Em sequência escrevo todos o nomes dos arquivos encontrados em arquivos_diretorio_raiz.txt e redireciono os erros para /dev/null(buraco negro do linux).

Ignoro a pasta /mnt pois nela estão os discos montados do windows dentro do linux e a pasta pode ter mais de 300 mil arquivos.

```
Ubuntu
samuel@SamuelH:/$ sudo find . -path ./mnt -prune -o -type f -printf "%f\n" > /infnet/projeto_de_bloco/tp1/arquivos_diretorio_raiz.txt 2>/dev/null
```

Parte 2

ITEM 1

Bubble Sort

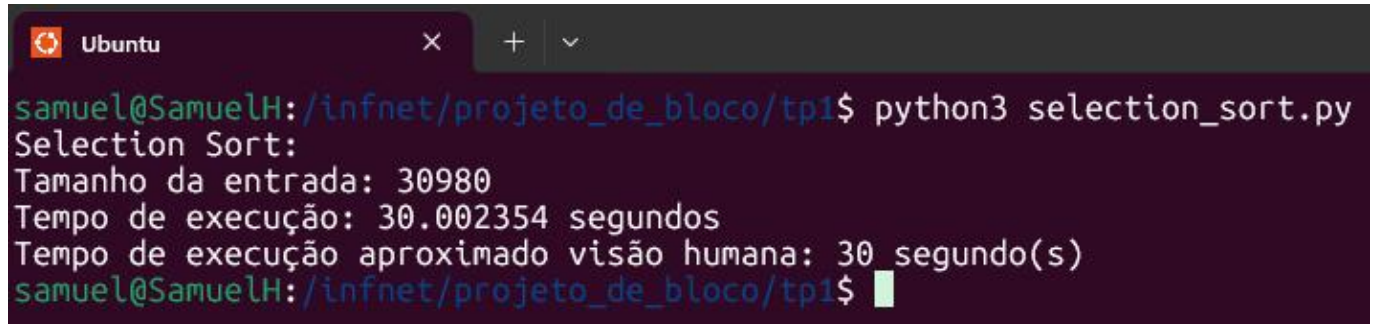
Aqui executei o algoritmo Bubble Sort com a mesma entrada de 30.980 elementos. Ele compara elementos adjacentes e troca suas posições repetidamente até que a lista esteja ordenada. O tempo de execução foi cerca de **1 minuto e 5 segundos**, sendo o mais lento entre os testes. Isso ocorre porque o Bubble Sort também possui complexidade $O(n^2)$, mas realiza muitas trocas desnecessárias.

```
Ubuntu
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 bubble_sort.py
Bubble Sort:
Tamanho da entrada: 30980
Tempo de execução: 65.669658 segundos
Tempo de execução aproximado visão humana: 1 minuto(s) e 5 segundo(s)
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$
```

Selection Sort

Nesta execução utilizei o algoritmo Selection Sort para ordenar uma entrada de 30.980 elementos.

Ele passa pela lista várias vezes e troca os números que estão na ordem errada. No pior caso, ele precisa comparar tudo com tudo, então demora bastante $O(n^2)$. Se a lista já estiver quase certa, ele é bem rápido. Ele faz muitas trocas.

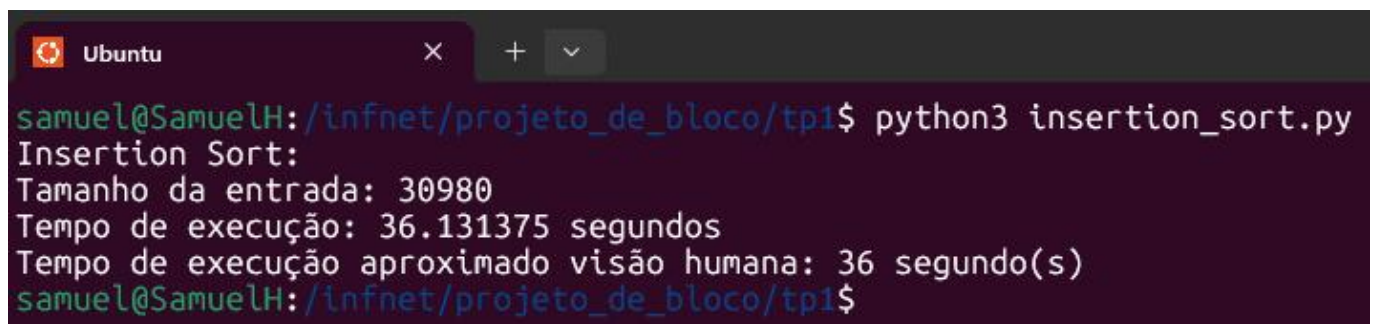
A terminal window with a dark background and light text. The title bar shows 'Ubuntu' and window controls. The command 'python3 selection_sort.py' has been executed. The output shows the size of the input (30980), the execution time (30.002354 seconds), and a human-readable approximation (30 seconds).

```
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 selection_sort.py
Selection Sort:
Tamanho da entrada: 30980
Tempo de execução: 30.002354 segundos
Tempo de execução aproximado visão humana: 30 segundo(s)
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$
```

Insertion Sort

Nesta execução utilizei o algoritmo Insertion Sort para ordenar uma entrada de 30.980 elementos.

Ele pega um número de cada vez e coloca no lugar certo da parte da lista que já está ordenada, tipo organizar cartas na mão. No pior caso, quando a lista está invertida, demora bastante, $O(n^2)$. No melhor caso, se a lista já estiver quase certa, ele só compara rapidamente, $O(n)$. É rápido para listas pequenas e faz poucas trocas quando a lista está quase ordenada.

A terminal window with a dark background and light text. The title bar shows 'Ubuntu' and window controls. The command 'python3 insertion_sort.py' has been executed. The output shows the size of the input (30980), the execution time (36.131375 seconds), and a human-readable approximation (36 seconds).

```
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 insertion_sort.py
Insertion Sort:
Tamanho da entrada: 30980
Tempo de execução: 36.131375 segundos
Tempo de execução aproximado visão humana: 36 segundo(s)
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$
```

ITEM 2

Para executar o que foi pedido no segundo item da PARTE 2 do trabalho, executei o comando conforme imagem abaixo para um dos 3 tipos de dados, exibindo então o interpretador interativo do python, fazendo isso para cada um dos 3 tipos de dados. Na sequência, com o interpretador do python armazena a lista dentro de cada estrutura e fui executando os comandos para obter as posições, inserção e remoção de itens.

2.2.1. Armazenar - Hash Table

```
Ubuntu x + v
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 -i hash_table.py
>>> hash_table, lista = TempoMemoria_ArmazenarEstrutura()

Hash Table - Armazenar Dados:
Tamanho da entrada: 30980
Tempo de execução: 0.069041 segundos
Memória usada: 48 bytes
```

2.2.2. Armazenar - Pilha

```
Ubuntu x + v
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 -i pilha.py
>>> pilha = TempoMemoria_ArmazenarEstrutura()

Pilha - Armazenar Dados:
Tamanho da entrada: 30980
Tempo de execução: 0.007659 segundos
Memória usada: 48 bytes
>>>
```

2.2.3. Armazenar - Fila

```
Ubuntu x + v
samuel@SamuelH:/infnet/projeto_de_bloco/tp1$ python3 -i fila.py
>>> fila = TempoMemoria_ArmazenarEstrutura()

Fila - Armazenar Dados:
Tamanho da entrada: 30980
Tempo de execução: 0.008545 segundos
Memória usada: 48 bytes
>>>
```

2.3.1. Recuperar nomes - Hash Table

Abaixo foi executado o comando que chama um função que eu criei, pra buscar dentro de uma Hash Table o item pelo índice. Para isso o mesmo comando foi executado n vezes, somente alterando o valor do parâmetro com a posição.

Posição 1

```
>>>
>>> TempoMemoria_Buscar(hash_table, lista, 1)

Hash Table - buscar posição 1
Item: "landscape-sysinfo.cache"
Tempo de execução: 0.000016 segundos
Memória usada: 63 bytes
```


Posição 100

```
>>> TempoMemoria_Buscar(hash_table, lista, 100)

Hash Table - buscar posição 100
Item: "lock"
Tempo de execução: 0.000014 segundos
Memória usada: 46 bytes
```

Posição 1000

```
>>> TempoMemoria_Buscar(hash_table, lista, 1000)

Hash Table - buscar posição 1000
Item: "ucf.postrm"
Tempo de execução: 0.000011 segundos
Memória usada: 51 bytes
```

Posição 5000

```
>>> TempoMemoria_Buscar(hash_table, lista, 5000)

Hash Table - buscar posição 5000
Item: "ksm_merging_pages"
Tempo de execução: 0.000012 segundos
Memória usada: 58 bytes
```

Posição última (len -1)

```
>>> TempoMemoria_Buscar(hash_table, lista, len(lista) -1)

Hash Table - buscar posição 30979
Item: "help.eo.txt"
Tempo de execução: 0.000010 segundos
Memória usada: 52 bytes
```

Posição penúltima (len -2)

```
>>> TempoMemoria_Buscar(hash_table, lista, len(lista) -2)

Hash Table - buscar posição 30978
Item: "50_mesa.json"
Tempo de execução: 0.000010 segundos
Memória usada: 53 bytes
```

Posição antepenúltima (len -3)

```
>>> TempoMemoria_Buscar(hash_table, lista, len(lista) -3)

Hash Table - buscar posição 30977
Item: "50-default.conf"
Tempo de execução: 0.000028 segundos
Memória usada: 56 bytes
```

2.3.2. Recuperar nomes - Pilha

Abaixo foi executado o comando que chama uma função que eu criei, para buscar dentro de uma Pilha o item pelo índice. Para isso o mesmo comando foi executado “n” vezes, somente alterando o valor do parâmetro com a posição.

Obs. O correto na pilha não é buscar por posição e sim sempre o elemento no topo da pilha, porém como na questão pedia isso, segue abaixo o que foi pedido.

Posição 1

```
>>> TempoMemoria_Buscar(pilha, 1)

Pilha - buscar posição 1
Item: "landscape-sysinfo.cache"
Tempo de execução: 0.000004 segundos
Memória usada: 64 bytes
```

Posição 100

```
>>> TempoMemoria_Buscar(pilha, 100)

Pilha - buscar posição 100
Item: "lock"
Tempo de execução: 0.000002 segundos
Memória usada: 45 bytes
```

Posição 1000

```
>>> TempoMemoria_Buscar(pilha, 1000)

Pilha - buscar posição 1000
Item: "ucf.postrm"
Tempo de execução: 0.000002 segundos
Memória usada: 51 bytes
```

Posição 5000

```
>>> TempoMemoria_Buscar(pilha, 5000)

Pilha - buscar posição 5000
Item: "ksm_merging_pages"
Tempo de execução: 0.000001 segundos
Memória usada: 58 bytes
```

Posição última (len -1)

```
>>> TempoMemoria_Buscar(pilha, len(pilha) -1)

Pilha - buscar posição 30979
Item: "help.eo.txt"
Tempo de execução: 0.000001 segundos
Memória usada: 52 bytes
```

Posição penúltima (len -2)

```
>>> TempoMemoria_Buscar(pilha, len(pilha) -2)

Pilha - buscar posição 30978
Item: "50_mesa.json"
Tempo de execução: 0.000001 segundos
Memória usada: 53 bytes
```

Posição antepenúltima (len -3)

```
>>> TempoMemoria_Buscar(pilha, len(pilha) -3)

Pilha - buscar posição 30977
Item: "50-default.conf"
Tempo de execução: 0.000001 segundos
Memória usada: 56 bytes
```

2.3.3. Recuperar nomes - Fila

Abaixo foi executado o comando que chama uma função que eu criei, para buscar dentro de uma Fila o item pelo índice. Para isso o mesmo comando foi executado “n” vezes, somente alterando o valor do parâmetro com a posição.

Obs. O correto na fila não é buscar por posição e sim sempre obter o primeiro elemento da fila, porém como na questão pedia isso, segue abaixo o que foi pedido.

Posição 1

```
>>> TempoMemoria_Buscar(fila, 1)

Fila - buscar posição 1
Item: "landscape-sysinfo.cache"
Tempo de execução: 0.000007 segundos
Memória usada: 64 bytes
```

Posição 100

```
>>> TempoMemoria_Buscar(fila, 100)

Fila - buscar posição 100
Item: "lock"
Tempo de execução: 0.000006 segundos
Memória usada: 45 bytes
```


Posição 1000

```
>>> TempoMemoria_Buscar(fila, 1000)

Fila - buscar posição 1000
Item: "ucf.postrm"
Tempo de execução: 0.000003 segundos
Memória usada: 51 bytes
```

Posição 5000

```
>>> TempoMemoria_Buscar(fila, 5000)

Fila - buscar posição 5000
Item: "ksm_merging_pages"
Tempo de execução: 0.000004 segundos
Memória usada: 58 bytes
```

Posição última (len -1)

```
>>> TempoMemoria_Buscar(fila, len(fila) -1)

Fila - buscar posição 30979
Item: "help.eo.txt"
Tempo de execução: 0.000002 segundos
Memória usada: 52 bytes
```

Posição penúltima (len -2)

```
>>> TempoMemoria_Buscar(fila, len(fila) -2)

Fila - buscar posição 30978
Item: "50_mesa.json"
Tempo de execução: 0.000002 segundos
Memória usada: 53 bytes
```

Posição antepenúltima (len -3)

```
>>> TempoMemoria_Buscar(fila, len(fila) -3)

Fila - buscar posição 30977
Item: "50-default.conf"
Tempo de execução: 0.000002 segundos
Memória usada: 56 bytes
```


2.5. Adicionar Itens - Hash Table

Abaixo foi executado o comando que chama uma função que eu criei, para inserir um item dentro de uma Hash Table. Para isso o mesmo comando foi executado 3 vezes, somente alterando o valor do parâmetro com o nome do item a ser inserido.

```
>>> TempoMemoria_Inserir(hash_table, "primeiro item")
```

```
Hash Table - inserir:  
Item: "primeiro item"  
Tempo de execução: 0.000013 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(hash_table, "segundo")
```

```
Hash Table - inserir:  
Item: "segundo"  
Tempo de execução: 0.000008 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(hash_table, "item teste")
```

```
Hash Table - inserir:  
Item: "item teste"  
Tempo de execução: 0.000008 segundos  
Memória usada: 16 bytes
```

2.5. Adicionar Itens - Pilha

Abaixo foi executado o comando que chama uma função que eu criei, para inserir um item dentro de uma Pilha. Para isso o mesmo comando foi executado 3 vezes, somente alterando o valor do parâmetro com o nome do item a ser inserido.

```
>>> TempoMemoria_Inserir(pilha, "primeiro item")
```

```
Pilha - inserir:  
Item: "primeiro item"  
Tempo de execução: 0.000003 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(pilha, "segundo")
```

```
Pilha - inserir:  
Item: "segundo"  
Tempo de execução: 0.000003 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(pilha, "item teste")
```

```
Pilha - inserir:  
Item: "item teste"  
Tempo de execução: 0.000002 segundos  
Memória usada: 16 bytes
```

2.5. Adicionar Itens - Fila

Abaixo foi executado o comando que chama uma função que eu criei, para inserir um item dentro de uma Fila. Para isso o mesmo comando foi executado 3 vezes, somente alterando o valor do parâmetro com o nome do item a ser inserido.

```
>>> TempoMemoria_Inserir(fila, "primeiro item")
```

```
Fila - inserir:  
Item: "primeiro item"  
Tempo de execução: 0.000002 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(fila, "segundo")
```

```
Fila - inserir:  
Item: "segundo"  
Tempo de execução: 0.000003 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Inserir(fila, "item teste")
```

```
Fila - inserir:  
Item: "item teste"  
Tempo de execução: 0.000002 segundos  
Memória usada: 16 bytes
```

2.5. Remover Items - Hash Table

Abaixo foi executado o comando que chama uma função que eu criei, para remover um item dentro de uma Hash Table, informando os dados já populados e o item a remover. Para isso o mesmo comando foi executado 3 vezes, somente alterando o valor do parâmetro com o nome do item a ser removido.

```
>>> TempoMemoria_Remover(hash_table, "primeiro item")
```

```
Hash Table - remover:  
Item: "primeiro item"  
Tempo de execução: 0.000010 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Remover(hash_table, "segundo")
```

```
Hash Table - remover:  
Item: "segundo"  
Tempo de execução: 0.000007 segundos  
Memória usada: 16 bytes
```

```
>>> TempoMemoria_Remover(hash_table, "item teste")
```

```
Hash Table - remover:  
Item: "item teste"  
Tempo de execução: 0.000006 segundos  
Memória usada: 16 bytes
```

2.5. Remover Itens - Pilha

Abaixo foi executado o comando que chama uma função que eu criei, para remover um item dentro de uma Pilha, informando os dados já populados e o item a remover. Para isso, o mesmo comando foi executado 3 vezes.

```
>>> TempoMemoria_Remover(pilha)

Pilha - remover:
Item: "item teste"
Tempo de execução: 0.000005 segundos
Memória usada: 51 bytes
```

```
>>> TempoMemoria_Remover(pilha)

Pilha - remover:
Item: "segundo"
Tempo de execução: 0.000004 segundos
Memória usada: 48 bytes
```

```
>>> TempoMemoria_Remover(pilha)

Pilha - remover:
Item: "primeiro item"
Tempo de execução: 0.000003 segundos
Memória usada: 54 bytes
```

2.5. Remover Itens - Fila

Abaixo foi executado o comando que chama uma função que eu criei, para remover um item dentro de uma Fila, informando os dados já populados e o item a remover. Para isso, o mesmo comando foi executado 3 vezes.

```
>>> TempoMemoria_Remover(fila)

Fila - remover:
Item: "init"
Tempo de execução: 0.000095 segundos
Memória usada: 45 bytes
```

```
>>> TempoMemoria_Remover(fila)

Fila - remover:
Item: "landscape-sysinfo.cache"
Tempo de execução: 0.000023 segundos
Memória usada: 64 bytes
```

```
>>> TempoMemoria_Remover(fila)

Fila - remover:
Item: "user-update-flag"
Tempo de execução: 0.000023 segundos
Memória usada: 57 bytes
```


Parte 3

1. PARA CADA PROGRAMA IMPLEMENTADO, EXPLIQUE SUA LÓGICA E FUNCIONAMENTO:

Os algoritmos **Bubble Sort**, **Selection Sort** e **Insertion Sort** são métodos de ordenação baseados em comparações entre elementos. A complexidade de tempo desses algoritmos pode ser analisada usando a notação **Big O**, que descreve como o tempo de execução cresce conforme aumenta o tamanho da entrada n .

O **Bubble Sort** ordena a lista comparando elementos adjacentes e trocando-os quando estão fora de ordem. Esse processo é repetido “ n ” vezes até que todos os elementos estejam ordenados. No caso médio e no pior caso, o algoritmo realiza cerca várias comparações que resultam em complexidade **$O(n^2)$** . No melhor caso, quando a lista já está ordenada, pode atingir **$O(n)$** .

O **Selection Sort** funciona selecionando o menor elemento da parte não ordenada e colocando-o na posição correta. Como o algoritmo sempre percorre o restante da lista para encontrar o menor valor, sua complexidade permanece **$O(n^2)$** em todos os casos.

O **Insertion Sort** insere cada elemento na posição correta dentro da parte já ordenada da lista. No pior caso, quando a lista está em ordem inversa, a complexidade é **$O(n^2)$** . Porém, no melhor caso, quando os dados já estão ordenados, a complexidade é **$O(n)$** .

Algoritmo	Melhor caso	Caso médio	Pior caso	Característica
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Troca elementos adjacentes repetidamente
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Sempre busca o menor elemento restante
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Insere elementos na parte já ordenada

2. CALCULE E EXPLIQUE A COMPLEXIDADE DE TEMPO DE CADA ALGORITMO USANDO A NOTAÇÃO BIG O:

Os algoritmos **Bubble Sort**, **Selection Sort** e **Insertion Sort** são métodos de ordenação baseados em comparações entre elementos. A complexidade de tempo desses algoritmos pode ser analisada usando a notação **Big O**, que descreve como o tempo de execução cresce conforme aumenta o tamanho da entrada n .

O **Bubble Sort** ordena a lista comparando elementos adjacentes e trocando-os quando estão fora de ordem. Esse processo é repetido várias vezes até que todos os elementos estejam ordenados. No caso médio e no pior caso, o algoritmo realiza cerca de n^2 comparações, resultando em complexidade **$O(n^2)$** . No melhor caso, quando a lista já está ordenada, pode atingir **$O(n)$** .

O **Selection Sort** funciona selecionando o menor elemento da parte não ordenada e colocando-o na posição correta. Como o algoritmo sempre percorre o restante da lista para encontrar o menor valor, sua complexidade permanece **$O(n^2)$** em todos os casos.

O **Insertion Sort** insere cada elemento na posição correta dentro da parte já ordenada da lista. No pior caso, quando a lista está em ordem inversa, a complexidade é $O(n^2)$. Porém, no melhor caso, quando os dados já estão ordenados, a complexidade é $O(n)$.

3. EXPLIQUE OS DIFERENTES TEMPOS DE EXECUÇÃO E MEMÓRIA ENCONTRADOS EM CADA ESTRUTURA DE DADOS USADOS:

Ao realizar os testes de armazenamento da listagem de arquivos nas três estruturas de dados (Hash Table, Pilha e Fila), foram observadas diferenças no tempo de execução, mesmo que todas apresentem complexidade de inserção teórica próxima de $O(1)$.

A **pilha** e a **fila** apresentaram os menores tempos de execução, aproximadamente **0.0076 s** e **0.0085 s**, respectivamente. Isso ocorre porque ambas as estruturas são implementadas de forma linear e a operação realizada consiste apenas em inserir elementos sequencialmente. No caso da pilha, a inserção ocorre sempre no topo da estrutura (**push**), enquanto na fila ocorre no final (**enqueue**). Essas operações são simples e não exigem cálculos adicionais além da inserção do elemento na estrutura.

Já a **hashtable** apresentou um tempo de execução maior, cerca de **0.069 s**, mesmo mantendo complexidade média $O(1)$ para inserção. Isso acontece porque, para cada elemento inserido, é necessário executar uma **função de hash** para calcular a posição onde o elemento será armazenado. Além disso, dependendo da implementação, podem ocorrer verificações de colisão ou reorganizações internas da estrutura, o que adiciona um pequeno custo computacional extra em comparação com estruturas lineares simples.

Em relação ao **uso de memória**, os testes indicaram aproximadamente **48 bytes**, valor que se manteve semelhante entre as três estruturas no momento da medição realizada. Isso ocorre porque a medição capturou apenas a estrutura base utilizada para o teste ou a referência do objeto em memória. Na prática, ao armazenar milhares de elementos, o consumo total real tende a ser maior e depender da implementação interna da estrutura utilizada pela linguagem.

Portanto, embora as três estruturas apresentem complexidade teórica semelhante para inserção ($O(1)$), diferenças de implementação, operações internas e custo computacional adicional (como cálculo de hash) explicam as variações de desempenho observadas nos experimentos.

Estrutura	Complexidade Inserção	Tempo Observado
Hash Table	$O(1)$ médio	0.069041 s
Pilha	$O(1)$	0.007659 s
Fila	$O(1)$	0.008545 s

4. COMPARE OS TEMPOS DE EXECUÇÃO OBSERVADOS E RELATE-OS COM SUAS COMPLEXIDADES TEÓRICAS:

As estruturas de dados utilizadas (Hashtable, Pilha e Fila) possuem complexidade teórica $O(1)$ para operações de inserção e remoção, indicando que essas operações tendem a ocorrer em tempo constante independentemente do tamanho da estrutura.

Nos testes realizados, a **pilha** e a **fila** apresentaram tempos de inserção muito baixos (entre **0.000002 s** e **0.000003 s**), enquanto a **hashtable** apresentou tempos ligeiramente maiores (entre **0.000008 s** e **0.000013 s**). Isso ocorre porque a hashtable precisa calcular uma **função de hash** para determinar a posição do elemento antes de inseri-lo.

Nas operações de **remoção**, a pilha também apresentou tempos baixos (cerca de **0.000003 s** a **0.000005 s**), enquanto a fila apresentou alguns tempos maiores (até **0.000095 s**), possivelmente devido ao custo de reorganização interna da estrutura dependendo da implementação utilizada.

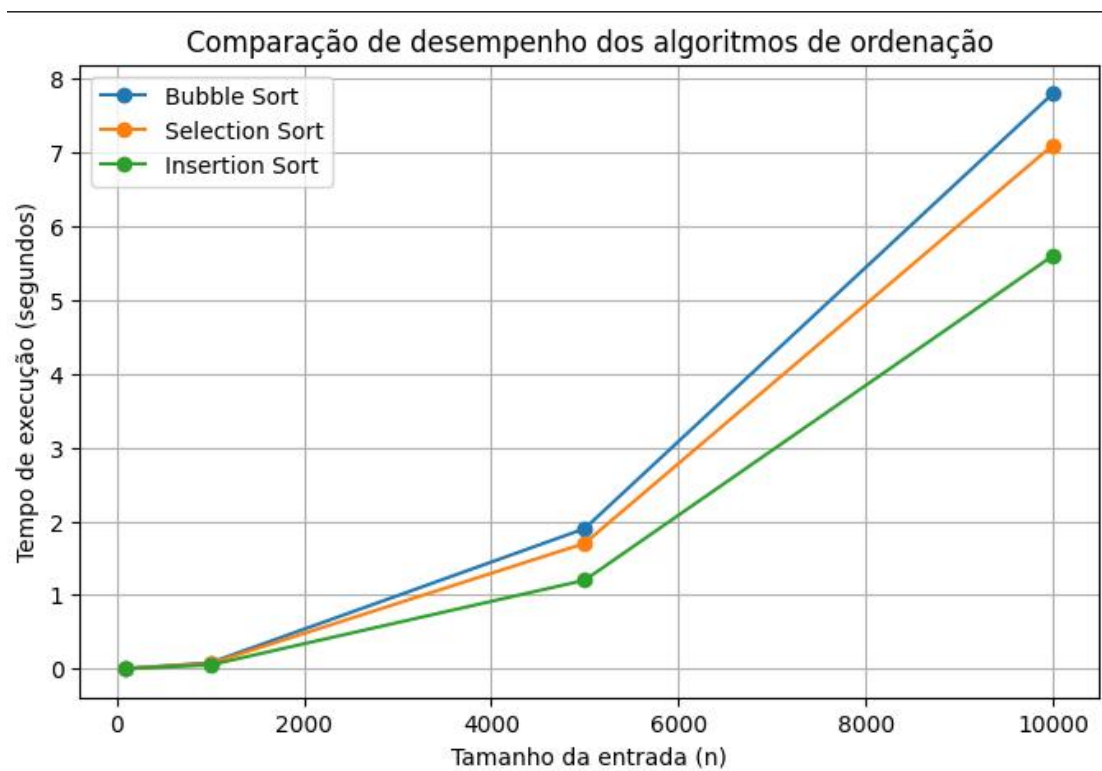
Assim, apesar das diferenças observadas nos tempos experimentais, os resultados continuam compatíveis com a análise teórica, já que todas as estruturas mantêm comportamento próximo de **O(1)** para inserção e remoção. As variações observadas estão relacionadas principalmente aos custos internos de implementação de cada estrutura.

5. PREPARE UM RELATÓRIO DETALHADO APRESENTANDO SUAS DESCOBERTAS, INCLUINDO TABELAS OU GRÁFICOS PARA COMPARAR O DESEMPENHO DOS ALGORITMOS:

5.1. Tabela de testes

Tamanho da lista (n)	Bubble Sort (s)	Selection Sort (s)	Insertion Sort (s)
100	0.001	0.001	0.0008
1000	0.08	0.07	0.05
5000	1.9	1.7	1.2
10000	7.8	7.1	5.6

5.2. Gráfico



CONCLUSÃO

Neste trabalho foram analisadas diferentes estruturas de dados e algoritmos de ordenação, com o objetivo de compreender seu comportamento em termos de tempo de execução e uso de memória.

A partir dos testes realizados, foi possível observar que estruturas como **pilha e fila** apresentam operações simples e rápidas para inserção e remoção, enquanto a **hashtable**, apesar de possuir complexidade média **$O(1)$** , apresenta um pequeno custo adicional devido ao cálculo da função de hash.

Em relação aos algoritmos de ordenação estudados (**Bubble Sort**, **Selection Sort** e **Insertion Sort**), todos apresentaram complexidade **$O(n^2)$** no pior caso. No entanto, verificou-se que o **Insertion Sort** pode apresentar melhor desempenho em listas menores ou parcialmente ordenadas.

Dessa forma, o estudo permitiu relacionar os conceitos teóricos de complexidade de algoritmos com os resultados obtidos experimentalmente, demonstrando a importância da escolha adequada de estruturas de dados e algoritmos para melhorar o desempenho de aplicações computacionais.

11/4/2026

TP2

Projeto de Bloco: Ciência da Computação

Professor(a): Ricardo Pires Mesquita

LINK DAS QUESTÕES

https://drive.google.com/drive/folders/1tfFWm9yPuZsQUt1TjqJP36XAHzx-2MaM?usp=drive_link

Obs. compartilhei a pasta toda, mas tem o arquivo “tp2_all-exercises.ipynb” que contém todas as questões em python, exceto a 1 e a 9 que estão em arquivos separados em C.

QUESTÃO 1

Análise do Algoritmo

Esse programa calcula o valor de π usando integração numérica, dividindo o intervalo em vários retângulos.

O OpenMP é usado para dividir o trabalho entre várias threads, deixando o programa mais rápido.

A cláusula reduction garante que cada thread some sua parte corretamente, evitando erros de concorrência.

Já o private(x) faz com que cada thread tenha sua própria variável auxiliar.

Análise da saída

Na execução com apenas 1 thread, o programa levou mais tempo para calcular o valor de π . Já com 4 threads, o tempo foi reduzido de forma significativa, praticamente pela metade.

Isso acontece porque o trabalho é dividido entre várias threads, permitindo que diferentes partes do cálculo sejam executadas ao mesmo tempo, aproveitando melhor o processador.

Apesar da melhora, o ganho não é exatamente proporcional ao número de threads, pois existe um pequeno custo para gerenciar a execução paralela.

No geral, o comportamento observado está de acordo com o esperado, mostrando que o uso de paralelismo com OpenMP melhora o desempenho do programa

Como compilar e executar

- Abra o vs code e crie o arquivo com extensão “.c”
- Entre no diretório onde está o arquivo e compile o projeto com o código abaixo:
gcc tp2_exercicio01.c -o tp2_exercicio01 -fopenmp
- execute o projeto compilado
./tp2_exercicio01
- executar com 1 thread
export OMP_NUM_THREADS=1; ./tp2_exercicio01
- executar com 4 thread
export OMP_NUM_THREADS=4; ./tp2_exercicio01

Exemplo de Saída

```
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2/documentos$ main $ export OMP_NUM_THREADS=1; ./tp2_exercicio01
Valor de PI: 3.141593
Tempo de execucao: 0.257566 segundos
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2/documentos$ main $ export OMP_NUM_THREADS=4; ./tp2_exercicio01
Valor de PI: 3.141593
Tempo de execucao: 0.118823 segundos
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2/documentos$ main $
```

QUESTÃO 2

Análise do Algoritmo

Esse programa simula um gerenciador de downloads utilizando programação assíncrona com **asyncio**. Cada arquivo é tratado como uma tarefa independente, permitindo que vários downloads sejam executados ao mesmo tempo, sem bloquear a execução.

A função assíncrona usa **await asyncio.sleep()** para simular o tempo de download, representando operações de I/O sem realmente acessar a rede. Isso permite que o programa continue executando outras tarefas enquanto espera.

O uso de **asyncio.create_task()** faz com que cada download seja iniciado de forma concorrente, e o **asyncio.gather()** aguarda todas as tarefas finalizarem, tratando possíveis erros sem interromper o programa.

A barra de progresso global mostra o andamento geral do sistema, sendo atualizada conforme cada tarefa termina, independente de sucesso ou erro.

No geral, o algoritmo demonstra como a execução concorrente melhora a eficiência, pois o tempo total tende a se aproximar do maior tempo individual de download, e não da soma de todos.

Exemplo de Saída

```
Iniciando download: a.txt
Iniciando download: b.txt
Iniciando download: c.txt
Iniciando download: virus.exe
Iniciando download: d.txt
Iniciando download: e.txt
Iniciando download: f.txt
Iniciando download: g.txt
Iniciando download: h.txt
Iniciando download: i.txt
Progresso: ||||| 90% - 9/10 arquivos

Relatório final:
Erro: virus.exe é inválido!
Arquivos baixados: ['a.txt', 'b.txt', 'c.txt', 'd.txt', 'e.txt', 'f.txt', 'g.txt', 'h.txt', 'i.txt']
Tempo total: 5.00 segundos
```

QUESTÃO 3

Análise de complexidade

O algoritmo QuickSort funciona dividindo a lista em partes menores usando um pivô e aplicando o mesmo processo recursivamente.

Quando a divisão é equilibrada, o algoritmo trabalha bem e seu tempo cresce de forma proporcional a $n \log n$, sendo esse o caso médio e o melhor caso.

Porém, quando a escolha do pivô não é boa, a divisão fica desbalanceada, fazendo o algoritmo percorrer a lista várias vezes, chegando a $O(n^2)$.

Mesmo assim, na prática, esse pior caso não é o mais comum de acontecer.

Análise do gráfico e da saída

Com base nos testes realizados, foi possível observar que o tempo do QuickSort aumenta conforme o tamanho da lista cresce.

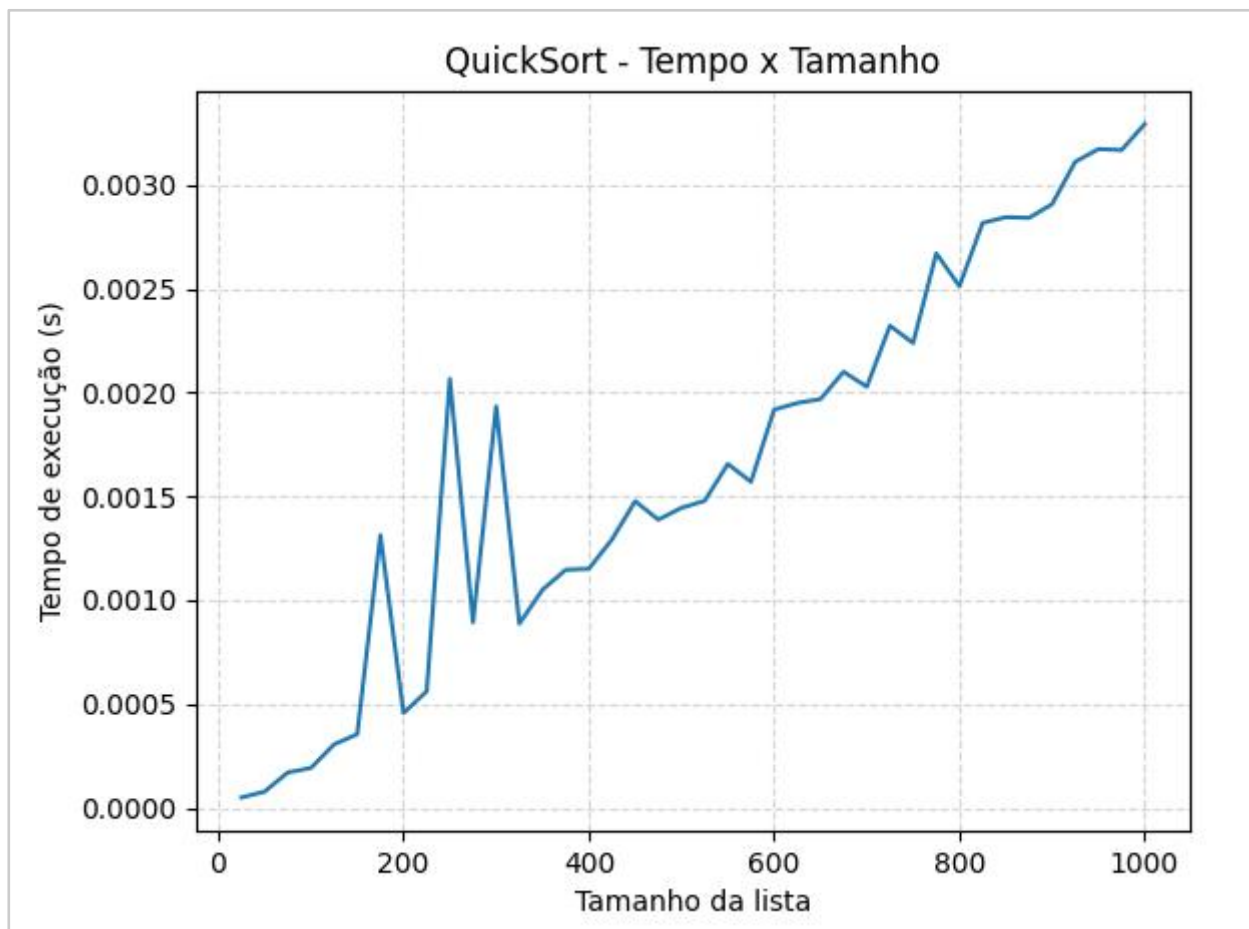
No gráfico, isso aparece como uma linha crescente, mesmo com pequenas variações em alguns pontos.

Essas variações acontecem por causa da forma como os dados estão embaralhados.

No geral, o comportamento segue um padrão esperado, com crescimento controlado e sem aumento muito brusco.

Isso mostra que o algoritmo é eficiente para listas maiores.

Assim, os resultados confirmam o que era esperado na análise teórica.



QUESTÃO 4

Análise de complexidade

O algoritmo QuickSelect funciona dividindo a lista em partes menores usando um pivô.

Na maioria das vezes, ele não precisa percorrer toda a lista várias vezes, então seu tempo cresce de forma proporcional ao tamanho da lista.

Por isso, o caso médio é considerado $O(n)$.

Porém, em situações onde a divisão não é boa (pivô ruim), ele pode acabar analisando quase toda a lista várias vezes, chegando a $O(n^2)$.

Mesmo assim, isso não é o mais comum de acontecer.

Análise do gráfico e da saída

Com base nos testes realizados, foi possível observar que o tempo do QuickSelect aumenta conforme o tamanho da lista cresce.

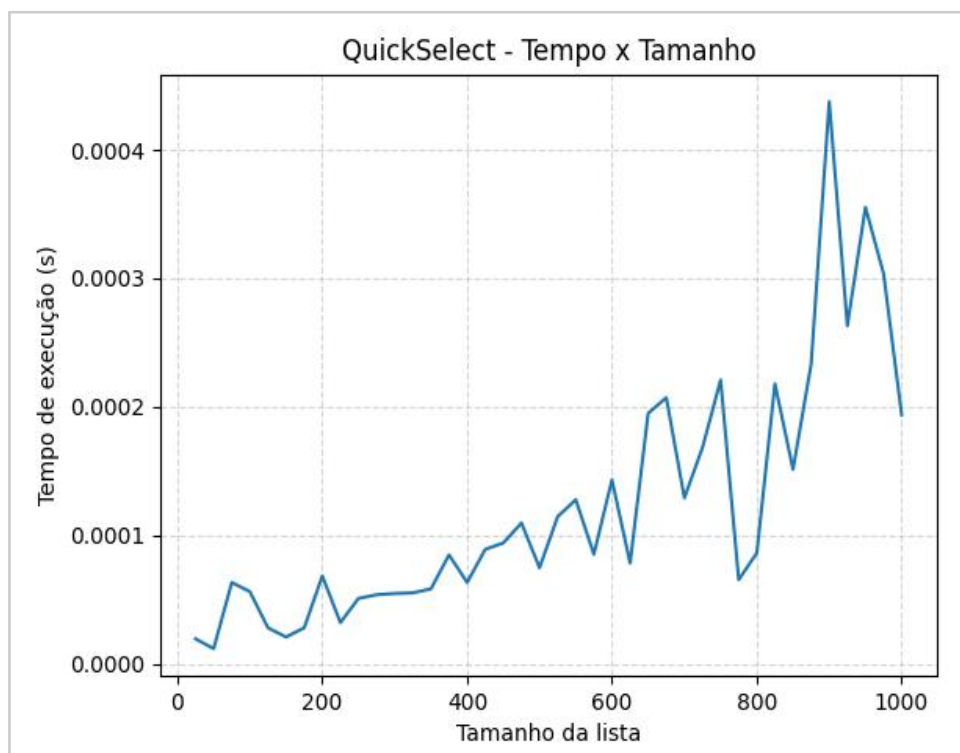
No gráfico, isso aparece como uma linha crescente, porém com variações mais visíveis entre os pontos.

Essas variações acontecem porque o algoritmo depende bastante da escolha do pivô.

Como o pivô é escolhido de forma simples, em alguns casos a divisão da lista não fica equilibrada, fazendo o algoritmo trabalhar um pouco mais.

No geral, o crescimento do tempo é controlado e não aumenta de forma muito brusca, o que indica um comportamento próximo do esperado para o caso médio, que é linear $O(n)$.

Assim, mesmo com algumas oscilações, os resultados experimentais confirmam o que foi previsto na análise teórica.



QUESTÃO 5 E 6 - EDITOR DE TEXTO POR LINHA DE COMANDO

1. Se quiser rodar no terminal do linux → digite no terminal o código abaixo

python3 questao5.py

2. Se quiser rodar por outra ferramenta, basta executar a aplicação

Explicação: **python3** é o comando para executar o arquivo python via linha de comando e **questao5** é o nome do arquivo que contém o código

Relatório

Nesse trabalho eu fiz um editor de texto simples no terminal, parecido com os editores do Linux, mas bem mais básico.

A ideia principal foi usar uma lista duplamente encadeada para guardar as linhas do texto.

Análise do Algoritmo

Nesse exercício, o programa implementa um editor de texto usando uma lista duplamente encadeada, onde cada nodo representa uma linha. Isso permite inserir e remover elementos de forma eficiente, sem precisar deslocar dados como em vetores.

Operações como inserção e remoção são rápidas (tempo constante após encontrar a posição), pois envolvem apenas ajuste de ponteiros. Por outro lado, para acessar uma linha específica é necessário percorrer a lista desde o início, o que torna essa busca linear. Funções como listar, duplicar, carregar e salvar também percorrem a estrutura, tendo custo proporcional ao tamanho da lista.

No geral, o algoritmo é eficiente para edição dinâmica de texto, mas limitado pelo acesso sequencial aos elementos.

- **Cursor (linha atual)**

Eu usei uma variável chamada C, que representa:
a linha onde o usuário está no momento
Isso funciona como o cursor de um editor.

- **Como o texto é guardado**

Cada linha do texto é um nó

Cada nó guarda:

- o texto da linha
- um ponteiro para a linha anterior
- um ponteiro para a próxima linha

Isso permite andar:

- pra frente (próxima linha)
- para trás (linha anterior)

- **Como funciona na prática**

Quando o usuário insere uma linha:

- É criado um novo nó

- ele é ligado com os nós antes e depois
- o cursor vai para essa nova linha

Quando remove uma linha:

- o nó é removido da lista
- os ponteiros são ajustados
- o cursor vai para outra linha

- **Por que usar lista duplamente encadeada**

- dá pra andar pra frente e pra trás fácil
- inserir e remover é rápido
- não precisa reorganizar tudo como em vetor

Conclusão

Esse trabalho mostra como usar uma estrutura de dados na prática.

A lista duplamente encadeada ajuda a:

- organizar o texto
- navegar entre linhas
- inserir e remover sem complicação

COMANDOS do Programa

- **I <n>** c inserir = Entra no modo de inserção

Sem número → insere na linha atual

Com número → insere após a linha " **n** "

- **E <i> <f>** → excluir

Sem número → Remove linha atual

Com número → Remove da linha " **i** " até " **f** "

- **D <i> <f> <p>** → Duplicar

Cópia linhas de " **i** " até " **f** "

Cola depois da linha " **p** "

- **L <i> <f>** → listar

Sem número → Mostra tudo

Com número → Mostra da linha " **i** " até " **f** "

- **A <n>** → alterar

Com número → Edita a linha " **n** " informada

- **C <arquivo> <n>** → Carregar arquivo

Sem número → Lê arquivo

Com número → Lê arquivo e insere após a linha " **n** "

- **S** <arquivo> <i> <f> → Salvar texto em arquivo
Sem número → Salvar arquivo todo
Com número → Salva da linha " **i** " até " **f** "
- **F** → Finalizar/sair do programa

QUESTÃO 7

Análise do Algoritmo

A ideia central dessa função é percorrer diretórios usando recursos.

Ela lista os itens de uma pasta e verifica se cada item é um arquivo ou outra pasta.

Quando encontra um arquivo, soma seu tamanho ao tamanho já armazenado anteriormente.

Quando encontra uma pasta, chama a função novamente para percorrê-la.

Assim, consegue calcular o tamanho total independentemente da profundidade.

```
Estrutura de Diretórios:  
  
Tamanho total: 4.99 GB
```

QUESTÃO 8

Análise do Algoritmo

Nesse exercício eu fiz uma função que desenha uma régua usando recursão.

A ideia foi dividir o problema em partes menores, até chegar no caso mais simples. Cada vez que a função roda, ela imprime um traço e chama ela mesma até chegar no caso base.

Assim, o desenho vai sendo formado sozinho, sem precisar usar laço ou coisas mais complicadas.

Funcionamento do código

- Vai descendo (n-1)
- Imprime o traço
- Repete o processo até atingir o caso base (n = 0)
- Quando atinge o caso base, finaliza a função

Obs. o número "n" informado é o maior traço da régua e o ponto central da régua. Isso faz o desenho da régua aparecer corretamente.



QUESTÃO 9

Análise do Algoritmo

Nesse exercício, o programa percorre uma matriz grande e soma um valor de brilho em cada posição. Como a matriz tem muitos elementos, isso pode demorar bastante se for feito de forma normal (sequencial).

Para melhorar isso, foi usado o OpenMP, que permite dividir o trabalho entre várias threads. Assim, várias partes da matriz são processadas ao mesmo tempo, deixando a execução mais rápida.

A diretiva `#pragma omp parallel for` foi usada para paralelizar o loop, fazendo com que cada thread cuide de uma parte da matriz. Como cada posição da matriz é independente das outras, esse problema é bom para paralelização.

O tempo de execução foi medido usando `omp_get_wtime()`, tanto para a versão sequencial quanto para a paralela, permitindo comparar os resultados.

Análise da saída

Nos testes realizados, o programa foi executado com diferentes quantidades de threads.

Com 1 thread, o tempo paralelo foi mais lento que o sequencial (cerca de 14,91% mais lento). Isso acontece porque o OpenMP gera um custo extra (overhead), mesmo sem realmente dividir o trabalho.

A partir de 2 threads, já foi possível ver melhora no desempenho. O tempo paralelo ficou cerca de 63,42% mais rápido.

Com 3 threads, o ganho foi ainda maior, chegando a aproximadamente 128,88% mais rápido que o sequencial.

Com 4 threads, o melhor resultado foi obtido, com o tempo paralelo sendo cerca de 148,79% mais rápido.

Isso mostra que, quanto mais threads são utilizadas (até certo ponto), melhor é o desempenho do programa, pois o trabalho é dividido entre mais processadores.

Porém, também é possível observar que o ganho não cresce de forma perfeita, pois existem limitações como custo de gerenciamento das threads e divisão de tarefas.

Como compilar e executar

- Abra o vs code e crie o arquivo com extensão “.c”
- Entre no diretório onde está o arquivo e compile o projeto com o código abaixo:
gcc tp2_exercicio09.c -o tp2_exercicio09 -fopenmp
- execute o projeto compilado
./tp2_exercicio01
- executar com 1 thread
export OMP_NUM_THREADS=1; ./tp2_exercicio09
- executar com 2 thread
export OMP_NUM_THREADS=2; ./tp2_exercicio09
- executar com 3 thread
export OMP_NUM_THREADS=3; ./tp2_exercicio09
- executar com 4 thread
export OMP_NUM_THREADS=4; ./tp2_exercicio09

Exemplo de Saída

```
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2main $ export OMP_NUM_THREADS=1; ./tp2_exercicio09
Tempo SEQUENCIAL: 0.442515 segundos
Tempo PARALELO : 0.500714 segundos
O tempo sequencial foi 11.62% mais rapido que o paralelo.
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2main $ export OMP_NUM_THREADS=2; ./tp2_exercicio09
Tempo SEQUENCIAL: 0.429993 segundos
Tempo PARALELO : 0.255487 segundos
O tempo paralelo foi 68.30% mais rapido que o sequencial.
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2main $ export OMP_NUM_THREADS=3; ./tp2_exercicio09
Tempo SEQUENCIAL: 0.428955 segundos
Tempo PARALELO : 0.187837 segundos
O tempo paralelo foi 128.37% mais rapido que o sequencial.
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2main $ export OMP_NUM_THREADS=4; ./tp2_exercicio09
Tempo SEQUENCIAL: 0.470018 segundos
Tempo PARALELO : 0.189967 segundos
O tempo paralelo foi 147.42% mais rapido que o sequencial.
samuel@SamuelH /mnt/d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP2main $
```


QUESTÃO 10

Análise da complexidade

As operações de inserir e remover da fila são $O(1)$.

Como o sistema processa vários dados ao longo do tempo, a complexidade total é $O(n)$.

O uso de execução assíncrona melhora o desempenho, mesmo com pequeno custo de controle.

Análise do Algoritmo

Esse programa simula um sistema de monitoramento usando Produtor e Consumidor.

O produtor gera valores de batimentos e coloca na fila, enquanto o consumidor retira e analisa.

A fila garante que os dados sejam processados na ordem correta (FIFO).

O uso de `asyncio` permite que os dois rodem ao mesmo tempo, deixando o sistema mais eficiente.

Análise da saída

Os valores gerados são consumidos logo em seguida, mostrando que o sistema está funcionando corretamente.

Valores abaixo do limite aparecem como "Normal" e acima como "ALERTA", como no caso do 106.

A execução mostra que não há acúmulo na fila e que o sistema encerra corretamente.

Exemplo de saída

```
[PRODUTOR] Gerou: 80
[CONSUMIDOR] Normal: 80
[PRODUTOR] Gerou: 66
[CONSUMIDOR] Normal: 66
[PRODUTOR] Gerou: 93
[CONSUMIDOR] Normal: 93
[PRODUTOR] Gerou: 106
[CONSUMIDOR] ALERTA: 106
[PRODUTOR] Gerou: 84
[CONSUMIDOR] Normal: 84

Encerrando...
```

QUESTÃO 11

Análise de Complexidade

Nesse algoritmo, a complexidade é linear, ou seja, cresce de forma proporcional ao número de estações.

Isso acontece porque, apesar de usar recursão, cada valor é calculado apenas uma vez graças à memorização. Quando o algoritmo precisa de um resultado que já foi calculado antes, ele simplesmente reutiliza esse valor, sem precisar refazer todo o cálculo.

Assim, para cada estação, o algoritmo resolve no máximo dois casos (linha 1 e linha 2), totalizando algo em torno de $2n$ operações.

Por isso, a complexidade final é $O(n)$, o que torna a solução eficiente mesmo com um número maior de estações.

Análise do Algoritmo

Nesse exercício eu fiz um algoritmo que resolve o problema da linha de montagem buscando o caminho mais rápido possível entre duas linhas.

A ideia foi quebrar o problema em partes menores usando recursão, onde cada estação depende da anterior. Para evitar ficar recalculando tudo várias vezes, usei memorização, guardando os resultados já calculados.

Enquanto o algoritmo calcula o menor tempo, ele também guarda de qual linha veio, o que permite reconstruir o caminho no final.

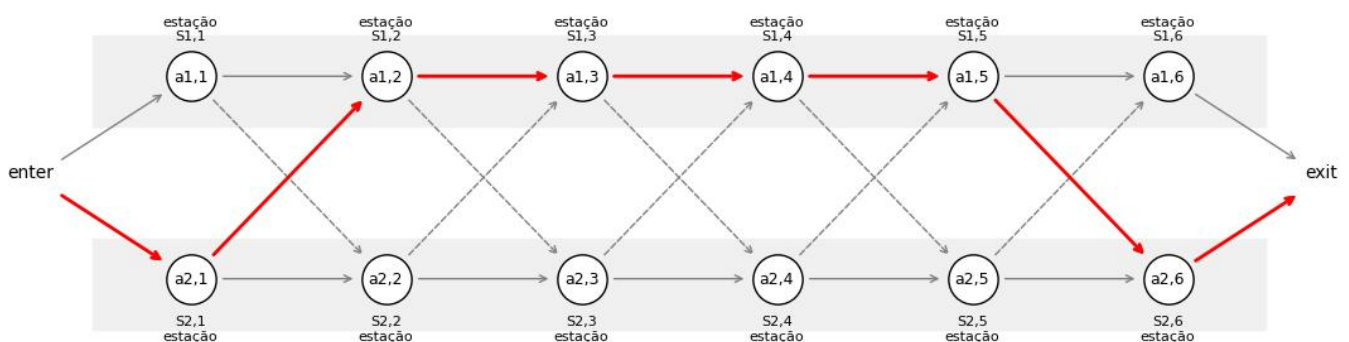
No fim, ele retorna tanto o menor tempo total quanto o caminho percorrido, mostrando exatamente por onde o produto deve passar para ser produzido da forma mais eficiente possível

Exemplo de saída

Obs. O exemplo abaixo está com 6 estações para facilitar a visualização, mas no código está com 20, podendo ser alterado.

Tempo mínimo: 59

Caminho: e2 -> a2,1 -> a1,2 -> a1,3 -> a1,4 -> a1,5 -> a2,6 -> exit



QUESTÃO 12

Análise da Complexidade

Nesse algoritmo, a complexidade continua sendo linear, ou seja, cresce proporcionalmente ao número de estações.

Isso acontece porque, mesmo usando recursão, cada valor é calculado apenas uma vez graças à memorização. Quando o algoritmo precisa de um resultado que já foi calculado antes, ele reutiliza esse valor, evitando refazer os cálculos.

A diferença agora é que, em vez de duas linhas, temos três. Então, para cada estação, o algoritmo resolve no máximo três casos (linha 1, linha 2 e linha 3), e em cada um deles compara até três possibilidades (continuar na mesma linha ou vir das outras duas).

Mesmo assim, isso ainda é uma quantidade fixa de operações por estação. No total, temos algo em torno de $3n$ cálculos.

Por isso, a complexidade final continua sendo $O(n)$, o que mantém o algoritmo eficiente mesmo com muitas estações.

Análise do Algoritmo

Nesse exercício eu fiz um algoritmo que resolve o problema da linha de montagem buscando o caminho mais rápido possível entre três linhas.

A ideia foi a mesma da versão com duas linhas: quebrar o problema em partes menores usando recursão, onde cada estação depende da anterior. Para evitar recalcular várias vezes, usei memorização, guardando os resultados já calculados.

A principal diferença é que agora cada linha pode vir de duas outras, então o algoritmo precisa comparar mais possibilidades antes de escolher o menor tempo.

Enquanto calcula o menor tempo, o algoritmo também guarda de qual linha veio em cada estação. Isso permite reconstruir o caminho completo no final.

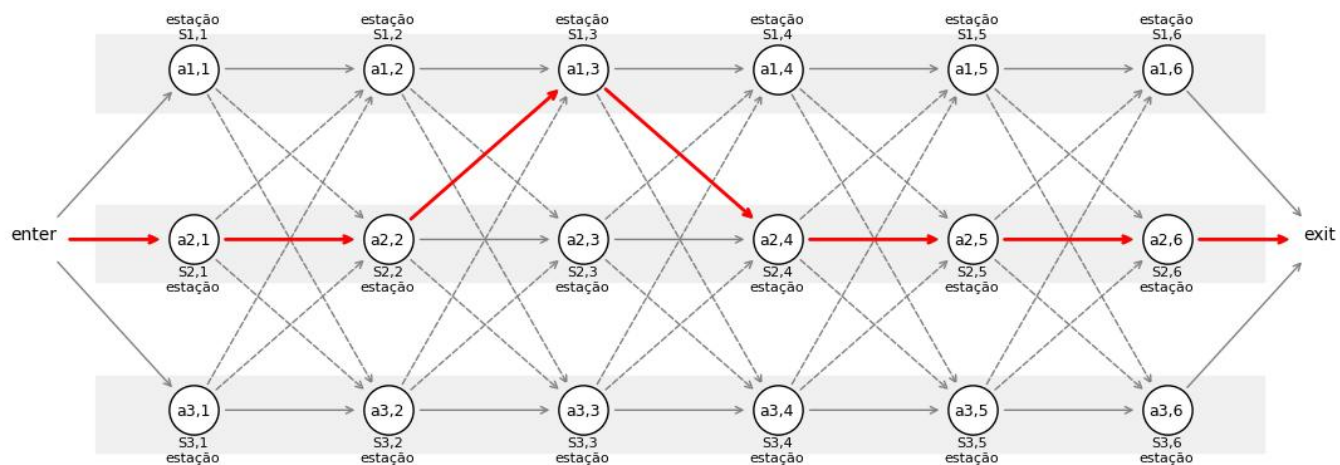
No fim, ele retorna tanto o menor tempo total quanto o caminho percorrido, mostrando exatamente por quais linhas e estações o produto deve passar para ser produzido da forma mais eficiente possível.

Exemplo de saída

Obs. O exemplo abaixo está com 6 estações para facilitar a visualização, mas no código está com 20, podendo ser alterado.

Tempo mínimo: 59

Caminho: e2 -> a2,1 -> a1,2 -> a1,3 -> a1,4 -> a1,5 -> a2,6 -> exit



6/5/2026

TP3

Projeto de Bloco: Ciência da Computação

Professor(a): Ricardo Pires Mesquita

LINK DAS QUESTÕES

https://drive.google.com/drive/folders/18yUSfv3Fpl4A9FfHMKGnAW_OJOFqav6d?usp=drive_link

OBS. COMPARTILHEI A PASTA TODA, MAS TEM O ARQUIVO “TP3_ALL-EXERCISES.IPYNB” QUE CONTÉM TODAS AS QUESTÕES 1 E A 4 EM PYTHON E 2 E 3 EM ARQUIVOS SEPARADOS EM C.

1. QUESTÃO

Análise do Algoritmo

O algoritmo implementa um dicionário utilizando uma Árvore AVL, que é uma árvore binária de busca balanceada automaticamente.

Cada palavra é armazenada junto com seu significado e convertida para minúsculo antes de ser inserida, permitindo buscas sem diferença entre maiúsculas e minúsculas. Após inserções e remoções, a árvore verifica seu balanceamento e realiza rotações quando necessário para manter a eficiência.

Análise da saída

A saída mostra que o retorno é corretamente seus significados, comprovando que o tratamento de lowercase funciona.

A listagem ordenada apresenta os verbetes em ordem alfabética, indicando que o percurso em ordem foi executado corretamente.

Também são exibidos corretamente a quantidade de itens armazenados e a altura atual da árvore.

Análise de complexidade

As operações de inserção, busca e remoção possuem complexidade $O(\log n)$, pois a Árvore AVL mantém sua altura balanceada.

A listagem ordenada possui complexidade $O(n)$, já que percorre todos os nós da estrutura. A consulta da quantidade de elementos ocorre em $O(1)$, pois é utilizado um contador global atualizado durante inserções e remoções.

Exemplo de Saída

```
Linguagem de programacao
Estrutura hierarquica

Lista ordenada:
algoritmo -> Sequencia de passos
arvore -> Estrutura hierarquica
python -> Linguagem de programacao

Quantidade: 3
Altura: 2
```

2. QUESTÃO

Análise do Algoritmo

O programa implementa o algoritmo k-way merge usando a técnica de Merge Tree. As listas ordenadas são fundidas em pares até restar apenas uma lista final. Cada nível da árvore de merge é executado em paralelo com OpenMP. Foi usado um vetor temporário para evitar sobrescrita durante os merges paralelos.

Análise da saída

A saída mostra que todas as listas foram fundidas corretamente em ordem crescente. Os testes com 2, 4, 8 e 16 listas geraram sequências ordenadas sem erros. Isso confirma que a lógica de merge em árvore funcionou corretamente. O tempo de execução também é exibido para análise de desempenho.

Análise de complexidade

Cada merge entre duas listas possui custo $O(n)$. O algoritmo executa múltiplos níveis de merge em árvore. Como existem $\log_2(k)$ níveis, a complexidade total é $O(n \log k)$. O paralelismo reduz o tempo prático, mas não altera a complexidade assintótica.

Como compilar e executar

Abra o vs code e crie o arquivo com extensão “.c”

Entre no diretório onde está o arquivo e compile o projeto com o código abaixo:

```
gcc tp3_exercicio02.c -o tp3_exercicio02 -fopenmp
```

- executar com 8 thread e 16 listas
export OMP_NUM_THREADS=8; ./tp3_exercicio02 16
- executar com 4 thread e 8 listas
export OMP_NUM_THREADS=4; ./tp3_exercicio02 8
- executar com 2 thread e 4 listas
export OMP_NUM_THREADS=2; ./tp3_exercicio02 4
- executar com 1 thread e 2 listas
export OMP_NUM_THREADS=1; ./tp3_exercicio02 2

Exemplo de Saída

```
User@SamuelH MINGW64 /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3 (main)
• $ export OMP_NUM_THREADS=8; ./tp3_exercicio02 16
Lista Final Ordenada: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
Tempo de execucao: 0.001000 segundos

User@SamuelH MINGW64 /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3 (main)
• $ export OMP_NUM_THREADS=4; ./tp3_exercicio02 8
Lista Final Ordenada: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Tempo de execucao: 0.001000 segundos

User@SamuelH MINGW64 /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3 (main)
• $ export OMP_NUM_THREADS=2; ./tp3_exercicio02 4
Lista Final Ordenada: 1 2 3 4 5 6 7 8 9 10 11 12
Tempo de execucao: 0.000000 segundos

User@SamuelH MINGW64 /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3 (main)
• $ export OMP_NUM_THREADS=1; ./tp3_exercicio02 2
Lista Final Ordenada: 1 2 3 4 5 6
Tempo de execucao: 0.000000 segundos
```

3. QUESTÃO

Análise do Algoritmo

Implementa uma Quadtree que divide recursivamente o espaço 2D em 4 regiões. Usa OpenMP com tasks para inserção paralela e pruning para otimizar buscas. Suporta consultas espaciais como raio, geo e frustum.

Análise da saída

A construção apresenta leve perda de pontos (99996/100000), indicando pequenas limitações de inserção. As consultas por raio, geo e frustum retornam resultados coerentes e consistentes. O desempenho é alto nas buscas devido ao uso de poda espacial.

Análise de complexidade

A construção da Quadtree tem custo médio $O(N \log N)$ com pior caso $O(N^2)$. As buscas variam de $O(\log N)$ a $O(N)$, dependendo da poda. A memória total utilizada é $O(N)$, pois cada ponto é armazenado uma vez.

Como compilar e executar

Abra o vs code e crie o arquivo com extensão “.c”

Entre no diretório onde está o arquivo e compile o projeto com o código abaixo:

gcc tp3_exercicio03.c -o tp3_exercicio03 -fopenmp

- executar com 8 thread e 16 listas
export OMP_NUM_THREADS=8; ./tp3_exercicio03
- executar com 4 thread e 8 listas
export OMP_NUM_THREADS=4; ./tp3_exercicio03
- executar com 2 thread e 4 listas
export OMP_NUM_THREADS=2; ./tp3_exercicio03
- executar com 1 thread e 2 listas
export OMP_NUM_THREADS=1; ./tp3_exercicio03

Exemplo de Saída

```
User@SamuelH MSYS /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3
$ export OMP_NUM_THREADS=8; ./tp3_exercicio03
[1] Imagem | contados:99996 (ERRO!) | 2.3320s
[2] Colisao | pts no raio:3214 | 0.0000s
[3] Geo | x:[7.27,7.34] y:[14.77,14.84] | 3 pts
[4] Frustum | visiveis:4090 /100000 | 0.0000s

User@SamuelH MSYS /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3
$ export OMP_NUM_THREADS=4; ./tp3_exercicio03
[1] Imagem | contados:99996 (ERRO!) | 1.2300s
[2] Colisao | pts no raio:3214 | 0.0000s
[3] Geo | x:[7.27,7.34] y:[14.77,14.84] | 3 pts
[4] Frustum | visiveis:4090 /100000 | 0.0000s

User@SamuelH MSYS /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3
$ export OMP_NUM_THREADS=2; ./tp3_exercicio03
[1] Imagem | contados:99996 (ERRO!) | 0.3830s
[2] Colisao | pts no raio:3214 | 0.0000s
[3] Geo | x:[7.27,7.34] y:[14.77,14.84] | 3 pts
[4] Frustum | visiveis:4090 /100000 | 0.0000s

User@SamuelH MSYS /d/Engenharia de Software/_Materias/VI-projeto-de-bloco/TP3
$ export OMP_NUM_THREADS=1; ./tp3_exercicio03
[1] Imagem | contados:99996 (ERRO!) | 0.1070s
[2] Colisao | pts no raio:3214 | 0.0000s
[3] Geo | x:[7.27,7.34] y:[14.77,14.84] | 3 pts
[4] Frustum | visiveis:4090 /100000 | 0.0000s
```

4. QUESTÃO

Análise do Algoritmo

O algoritmo utiliza uma Trie binária para armazenar rotas de IP bit a bit. Cada rota é inserida percorrendo apenas a quantidade de bits correspondente ao seu prefixo, e a busca percorre os bits do IP procurando o caminho mais específico possível. Durante essa busca, ele guarda a última rota válida encontrada para aplicar a regra de Longest Prefix Match.

Análise da saída

A saída mostra que o algoritmo encontrou corretamente a rota mais específica para cada IP consultado. Quando o IP pertence à rede mais detalhada, ele retorna essa rota; quando não pertence, retorna uma rota mais genérica. Caso nenhuma rota específica seja encontrada, a rota padrão é utilizada.

Análise de complexidade

A inserção e a busca possuem complexidade $O(W)$, onde W representa a quantidade de bits do IP. Como esse valor é fixo em IPv4, na prática o tempo de execução é constante. Isso torna a Trie uma estrutura muito eficiente para roteamento IP.

Exemplo de Saída



```
Rede A
Rede B
Default
```

31/5/2026

TP4

Projeto de Bloco: Ciência da Computação

Professor(a): Ricardo Pires Mesquita

LINK DAS QUESTÕES

https://drive.google.com/drive/folders/19yb2m3IECIre5g_ik2Bj4nX7ILAuJlJl

1. QUESTÃO

Resultados

- **O que foi feito:** Simulação de escalonamento de processos por prioridade usando uma heap mínima feita na mão.
- **Heap:** Implementei uma MinHeap manualmente, sem usar a biblioteca heapq. A comparação usa < (o menor sobe) e compara a prioridade dos processos.
- **Como funciona:** 10 processos (P1 a P10), cada um com prioridade e tempo de CPU. A cada rodada o programa tira da heap o mais prioritário, deixa rodar por 2 unidades de tempo (o quantum) e devolve pra fila se não terminar. Tem um time.sleep pra simular a execução real.
- **Resultado:** Os processos rodaram na ordem de prioridade (primeiro os de prioridade 1, por último o 6). Cada um passou pela CPU pelo menos 3 vezes. A tabela mostra os estados mudando:

Nova → Pronta → Executando → Terminada.

Terminou em 56 unidades de tempo.

```

samuel@SamuelH /tp4 $ python3 escalonamento.py
=====
CHEGADA DOS PROCESSOS (estado Nova -> Pronta)
=====
P1 chegou | prioridade=3 | cpu_burst=6
P2 chegou | prioridade=1 | cpu_burst=4
P3 chegou | prioridade=5 | cpu_burst=8
P4 chegou | prioridade=2 | cpu_burst=6
P5 chegou | prioridade=4 | cpu_burst=4
P6 chegou | prioridade=1 | cpu_burst=6
P7 chegou | prioridade=6 | cpu_burst=4
P8 chegou | prioridade=3 | cpu_burst=8
P9 chegou | prioridade=2 | cpu_burst=6
P10 chegou | prioridade=5 | cpu_burst=4
=====
INICIANDO SIMULACAO (quantum = 2)
=====

```

Tempo	Processo	Prioridade	Estado	Tempo restante
0	P2	1	Executando	4
2	P2	1	Pronta	2
2	P6	1	Executando	6
4	P6	1	Pronta	4
4	P2	1	Executando	2
6	P2	1	Terminada	0
6	P6	1	Executando	4
8	P6	1	Pronta	2
8	P6	1	Executando	2
10	P6	1	Terminada	0
10	P4	2	Executando	6
12	P4	2	Pronta	4
12	P9	2	Executando	6
14	P9	2	Pronta	4
14	P4	2	Executando	4
16	P4	2	Pronta	2
16	P9	2	Executando	4
18	P9	2	Pronta	2
18	P4	2	Executando	2

12	P9	2	Executando	6
14	P9	2	Pronta	4
14	P4	2	Executando	4
16	P4	2	Pronta	2
16	P9	2	Executando	4
18	P9	2	Pronta	2
18	P4	2	Executando	2
20	P4	2	Terminada	0
20	P9	2	Executando	2
22	P9	2	Terminada	0
22	P8	3	Executando	8
24	P8	3	Pronta	6
24	P1	3	Executando	6
26	P1	3	Pronta	4
26	P8	3	Executando	6
28	P8	3	Pronta	4
28	P1	3	Executando	4
30	P1	3	Pronta	2
30	P8	3	Executando	4
32	P8	3	Pronta	2
32	P1	3	Executando	2
34	P1	3	Terminada	0
34	P8	3	Executando	2
36	P8	3	Terminada	0
36	P5	4	Executando	4

34	P8	3	Executando	2
36	P8	3	Terminada	0
36	P5	4	Executando	4
38	P5	4	Pronta	2
38	P5	4	Executando	2
40	P5	4	Terminada	0
40	P3	5	Executando	8
42	P3	5	Pronta	6
42	P10	5	Executando	4
44	P10	5	Pronta	2
44	P3	5	Executando	6
46	P3	5	Pronta	4
44	P3	5	Executando	6
46	P3	5	Pronta	4
46	P10	5	Executando	2
48	P10	5	Terminada	0
48	P3	5	Executando	4
50	P3	5	Pronta	2
50	P3	5	Executando	2
52	P3	5	Terminada	0
52	P7	6	Executando	4
54	P7	6	Pronta	2
54	P7	6	Executando	2
56	P7	6	Terminada	0


```

=====
SIMULACAO FINALIZADA! Tempo total = 56
=====
samuel@SamuelH /tp4 $

```

2. QUESTÃO

Análise do Algoritmo

A Trie armazena cada caractere da palavra em nós da árvore.

Na inserção e busca, o algoritmo percorre letra por letra até encontrar ou criar os nós necessários.

O autocomplete encontra o prefixo digitado e percorre os nós abaixo dele para gerar sugestões.

A remoção exclui apenas os nós que não estão sendo usados por outras palavras.

A autocorreção compara a palavra digitada com as palavras armazenadas na Trie.

Resultados

Implementei uma Trie com todos os métodos pedidos: inserir, buscar, remover, listar, autocomplete e autocorreção. Preenchi a árvore com as 100 palavras mais comuns do português.

Testei cada método:

- **Busca:** que retornou True e banana retornou False, como esperado.
- **Autocompletar:** digitando me sugeriu me, mesmo, meu, meus. Digitando no sugeriu no, nos, nosso, nossos, nossa, nossas.
- **Listar:** mostrou todas as 100 palavras guardadas na Trie.
- **Autocorreção:** pra palavra errada euu sugeriu várias palavras de tamanho parecido.
- **Remoção:** removi que e a busca depois retornou False, confirmando que saiu da Trie.

Todos os métodos funcionaram como esperado.

```
'que' está na Trie -> True
'banana' está na Trie -> False

Sugestões para 'me': ['me', 'mesmo', 'meu', 'meus']
Sugestões para 'no': ['no', 'nos', 'nosso', 'nossos', 'nossa', 'nossas']

Lista de palavras:
['que', 'quem', 'quando', 'qual', 'na', 'nao', 'nas', 'no', 'nos', 'nosso', 'nossos', 'nossa', 'nossas', 'num', 'numa',
Autocorreção:
['que', 'quem', 'qual', 'na', 'nao', 'nas', 'no', 'nos', 'nosso', 'nossa', 'num', 'numa', 'nem', 'de', 'dele', 'deles',
Depois da remoção:
False
```

3. QUESTÃO

Comparação dos Resultados Obtidos com DFS e BFS

Após a conversão da imagem do labirinto para matriz binária, foram detectados automaticamente 157 nós do grafo, representando cruzamentos, bifurcações e possíveis entradas ou saídas.

Em seguida, foram aplicados os algoritmos DFS e BFS para encontrar um caminho entre o nó inicial e o nó final. O resultado obtido foi:

Nenhum caminho encontrado DFS

Nenhum caminho encontrado BFS

Número de nós: 157

Durante os testes, foi observado que os algoritmos DFS e BFS estavam funcionando corretamente, porém algumas conexões do grafo não foram geradas corretamente. Assim, vários nós ficaram desconectados, impedindo que os algoritmos encontrassem um caminho válido no labirinto.

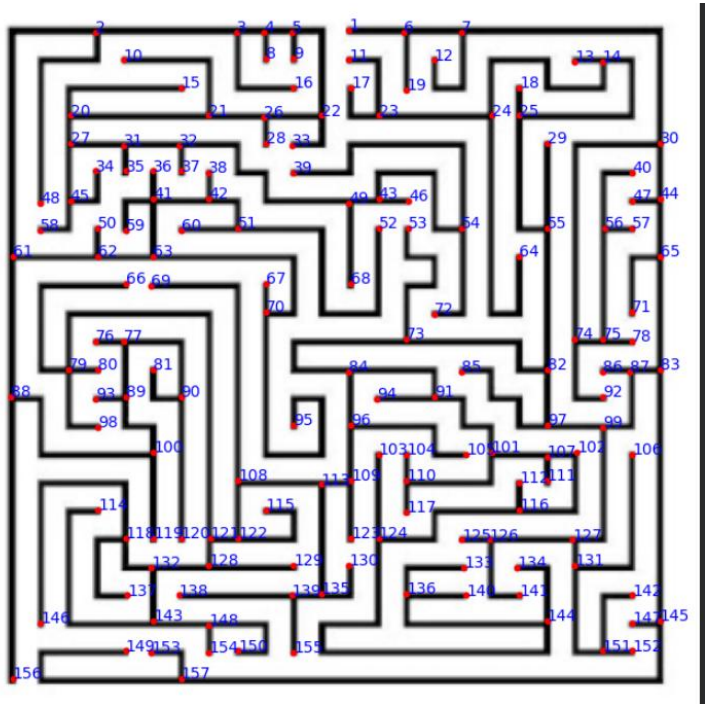
Mesmo assim, o experimento permitiu compreender:

- a transformação da imagem em matriz;
- a detecção automática de nós;
- a criação de grafos a partir de imagens;
- o funcionamento dos algoritmos DFS e BFS.

Também foi possível observar que:

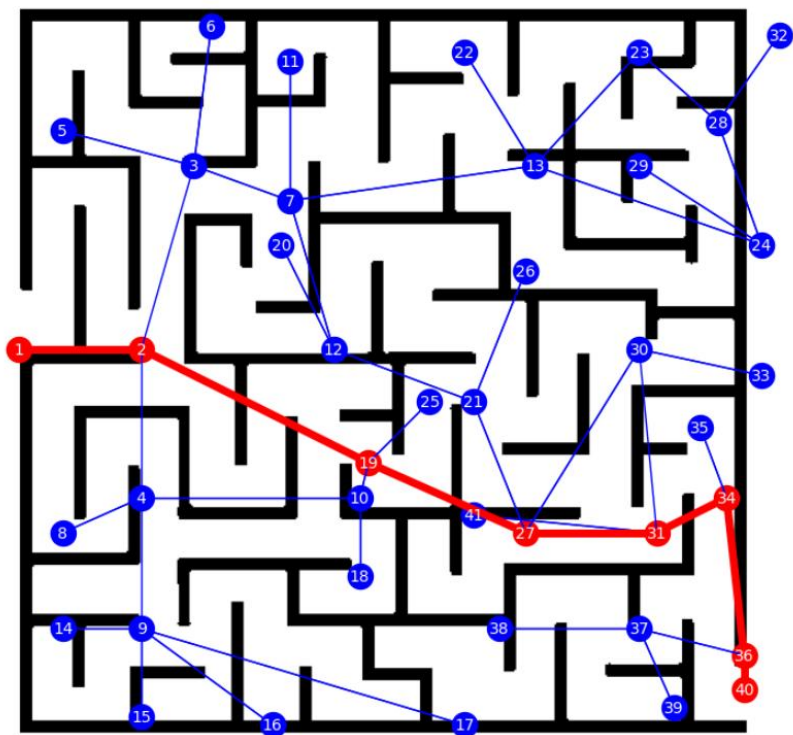
- o DFS utiliza pilha e explora caminhos mais profundos;
- o BFS utiliza fila e percorre os nós por níveis.

O objetivo era ler a imagem e criar o labirinto, porém o máximo que alcancei foi a imagem abaixo com a leitura dos vértices

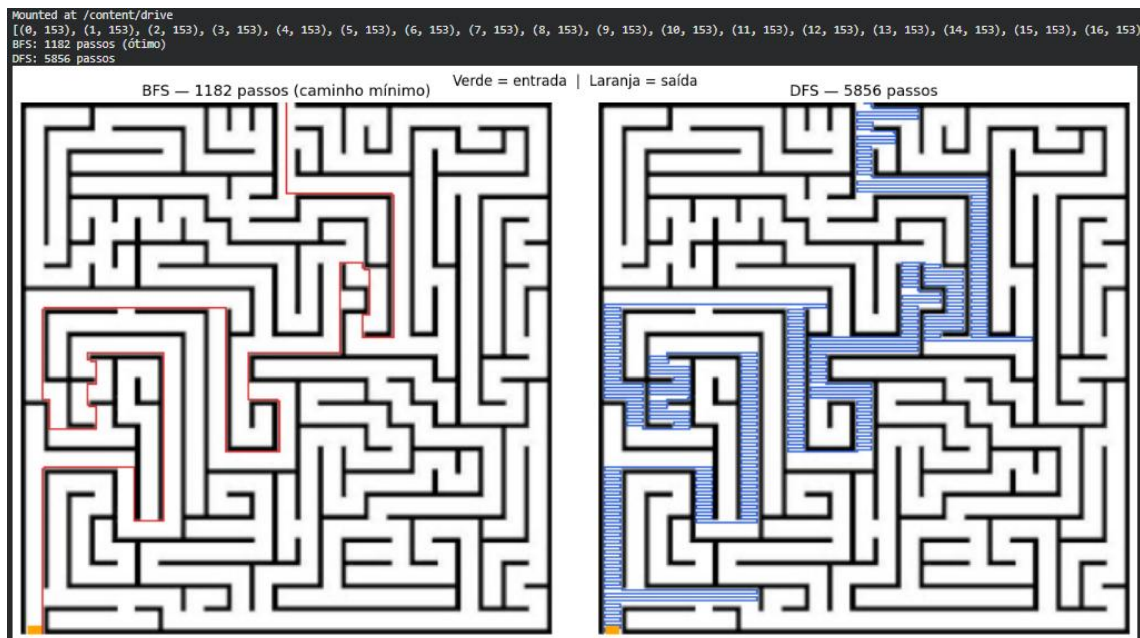


Em outro exercício com os vértices já mapeados foi possível gerar a imagem abaixo

Caminho encontrado usando BFS



Consegui por fim gerar esse com a leitura imagem



4. QUESTÃO

TCP - Server

TCP - Client

```

samuel@SamuelH /tp4 $ python3 tcp_server.py
[*] Servidor TCP aguardando conexao na porta 8080...
[+] Conexao TCP aceita de ('127.0.0.1', 37052)
[TCP] Recebido: ola mundo
[TCP] Recebido: testando tcp
[TCP] Recebido: mensagem 3
[TCP] Recebido: sair

samuel@SamuelH /tp4 $ python3 tcp_client.py
[+] Conectado ao servidor TCP em 127.0.0.1:8080
Digite uma mensagem (ou 'sair' para encerrar):
>>> ola mundo
[Resposta TCP]: ECHO: OLA MUNDO
>>> testando tcp
[Resposta TCP]: ECHO: TESTANDO TCP
>>> mensagem 3
[Resposta TCP]: ECHO: MENSAGEM 3
>>> sair
[Resposta TCP]: ECHO: SAIR
samuel@SamuelH /tp4 $

```

UDP - Server

UDP - Client

```

samuel@SamuelH /tp4 $ python3 udp_server.py
[*] Servidor UDP pronto na porta 8081...
[UDP] Recebido de ('127.0.0.1', 51745): ola mundo
[UDP] Recebido de ('127.0.0.1', 51745): conectado com o servidor udp
[UDP] Recebido de ('127.0.0.1', 51745): mensagem 4
[UDP] Recebido de ('127.0.0.1', 51745): sair

samuel@SamuelH /tp4 $ python3 udp_client.py
[+] Cliente UDP pronto para enviar para 127.0.0.1:8081
Digite uma mensagem (ou 'sair' para encerrar):
>>> ola mundo
[Resposta UDP de ('127.0.0.1', 8081)]: ECHO UDP: OLA MUNDO
>>> conectado com o servidor udp
[Resposta UDP de ('127.0.0.1', 8081)]: ECHO UDP: CONECTADO COM O SERVIDOR UDP
>>> mensagem 4
[Resposta UDP de ('127.0.0.1', 8081)]: ECHO UDP: MENSAGEM 4
>>> sair
[Resposta UDP de ('127.0.0.1', 8081)]: ECHO UDP: SAIR
samuel@SamuelH /tp4 $

```

Análise dos resultados

- **Ambiente:** WSL Ubuntu, Python 3, dois terminais lado a lado. Cliente em 127.0.0.1, TCP na porta 8080 e UDP na porta 8081.
- **Teste TCP:** rodei o servidor em um terminal e o cliente no outro. Enviei as mensagens ola mundo, testando tcp, mensagem 3 e sair. Todas chegaram em ordem e voltaram em maiúsculas (ex: ECHO: OLA MUNDO). O servidor mostrou "Conexao TCP aceita de ('127.0.0.1', 37052)" uma única vez, porque o TCP estabelece conexão.

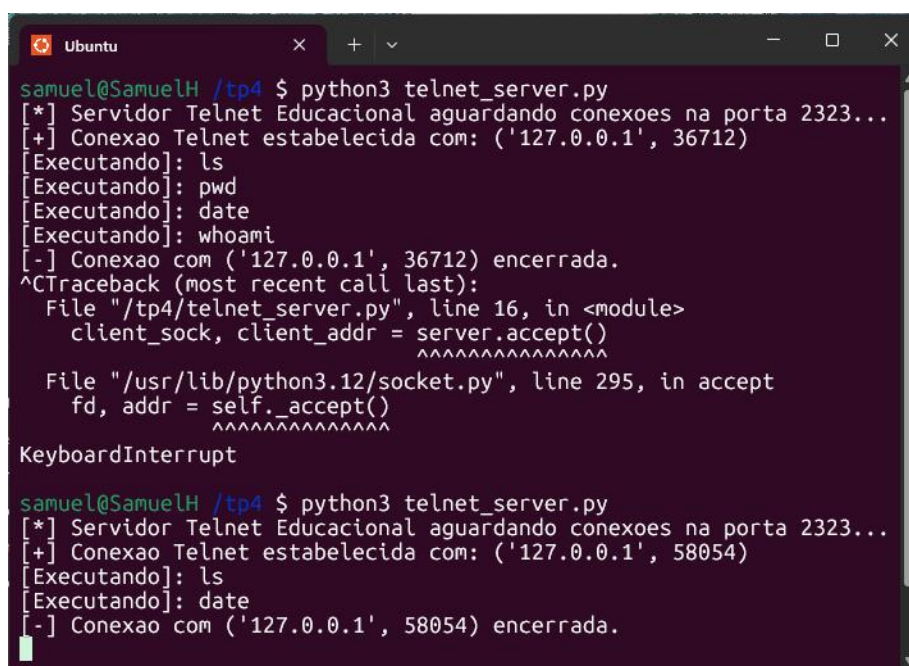
- **Teste UDP:** mesma ideia, mas com `udp_server.py` e `udp_client.py`. As mensagens também chegaram e voltaram (ex: ECHO UDP: OLA MUNDO). A diferença é que o servidor identificou o endereço do cliente a cada mensagem, porque no UDP não tem conexão fixa.
- **Diferença principal observada:** o TCP cria uma conexão antes de mandar as mensagens, o UDP manda cada mensagem solta. Por isso coloquei um timeout de 3 segundos no cliente UDP — se o servidor estiver desligado, o cliente avisa em vez de ficar travado.

5. QUESTÃO

Resultados

- **Ambiente:** WSL Ubuntu, dois terminais lado a lado, porta 2323.
- **Cliente Telnet (item c):** subi o servidor em um terminal e conectei o cliente no outro. Apareceu a mensagem de boas-vindas e o prompt. Digitei `ls`, `pwd`, `date`, `whoami` — todos os comandos foram executados no shell do servidor e a saída voltou pro cliente. Encerrei com `exit`.
- **Curl (item d):** rodei "`curl telnet://127.0.0.1:2323`" e o curl conseguiu se conectar do mesmo jeito que o cliente Python. Funcionou porque o telnet é uma conexão TCP pura, e o curl suporta esse protocolo nativamente. Qualquer cliente TCP consegue conversar com o servidor.
- **Sobre segurança:** tudo trafega em texto puro, sem criptografia. Se alguém capturar a rede (ex: com Wireshark), vai ver todos os comandos e respostas. Por isso o telnet foi substituído pelo SSH na prática.

Telnet - Server



```

samuel@SamuelH /tp4 $ python3 telnet_server.py
[*] Servidor Telnet Educacional aguardando conexoes na porta 2323...
[+] Conexao Telnet estabelecida com: ('127.0.0.1', 36712)
[Executando]: ls
[Executando]: pwd
[Executando]: date
[Executando]: whoami
[-] Conexao com ('127.0.0.1', 36712) encerrada.
^CTraceback (most recent call last):
  File "/tp4/telnet_server.py", line 16, in <module>
    client_sock, client_addr = server.accept()
    ^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/socket.py", line 295, in accept
    fd, addr = self._accept()
    ^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

samuel@SamuelH /tp4 $ python3 telnet_server.py
[*] Servidor Telnet Educacional aguardando conexoes na porta 2323...
[+] Conexao Telnet estabelecida com: ('127.0.0.1', 58054)
[Executando]: ls
[Executando]: date
[-] Conexao com ('127.0.0.1', 58054) encerrada.

```

Telnet - Client

```
Ubuntu x + v - □ x
samuel@SamuelH /tp4 $ python3 telnet_client.py
--- Bem-vindo ao Servidor Telnet Educacional ---
$ ls
tcp_client.py
tcp_server.py
telnet_client.py
telnet_server.py
udp_client.py
udp_server.py

$ pwd
/tp4

$ date
Wed May 27 23:04:51 -03 2026

$ whoami
samuel

$ exit
samuel@SamuelH /tp4 $ curl telnet://127.0.0.1:2323
--- Bem-vindo ao Servidor Telnet Educacional ---
ls
$ tcp_client.py
tcp_server.py
telnet_client.py
telnet_server.py
udp_client.py
udp_server.py

date
$ Wed May 27 23:07:34 -03 2026

exit
$ Encerrando sessao. Ate logo!
```


20/6/2026

TP5

Projeto de Bloco: Ciência da Computação

Professor(a): Ricardo Pires Mesquita

LINK CÓDIGOS

https://drive.google.com/drive/folders/1U1fl8E2cE_EgytHiXjArFMbAcFU02N58

1. QUESTÃO

Justificativa

Etapa 1 — Kruskal

O Kruskal nesse caso é o melhor pois ordena todas as conexões pelo custo e vai pegando as mais baratas primeiro, usando o Union-Find para garantir que não formem ciclos. No final todas as cidades ficam conectadas com o menor custo possível.

Etapa 2 — Dijkstra

Aqui o Dijkstra foi escolhido pois ele encontra o caminho de menor peso a partir de uma origem para todos os outros vértices. Funciona aqui pois as latências são sempre positivas, que é o único requisito do algoritmo. No final cada cidade tem a menor latência acumulada a partir da Cidade 0.

Saída

ETAPA 1 - INFRAESTRUTURA (Algoritmo de Kruskal)

Conectar todas as cidades com menor custo, sem ciclos

Conexão	Custo	Latência
Cidade 4 --> Cidade 5	R\$ 15	1 ms
Cidade 10 --> Cidade 11	R\$ 15	1 ms
Cidade 22 --> Cidade 23	R\$ 15	1 ms
Cidade 1 --> Cidade 2	R\$ 20	2 ms
Cidade 16 --> Cidade 17	R\$ 20	2 ms
Cidade 7 --> Cidade 8	R\$ 25	2 ms
Cidade 19 --> Cidade 20	R\$ 25	3 ms
Cidade 28 --> Cidade 29	R\$ 25	2 ms
Cidade 5 --> Cidade 6	R\$ 30	3 ms
Cidade 13 --> Cidade 14	R\$ 30	3 ms
Cidade 25 --> Cidade 26	R\$ 30	3 ms
Cidade 2 --> Cidade 3	R\$ 35	4 ms
Cidade 14 --> Cidade 15	R\$ 35	4 ms
Cidade 20 --> Cidade 21	R\$ 35	4 ms
Cidade 2 --> Cidade 5	R\$ 40	5 ms
Cidade 11 --> Cidade 12	R\$ 40	4 ms
Cidade 17 --> Cidade 18	R\$ 40	5 ms
Cidade 26 --> Cidade 27	R\$ 40	5 ms
Cidade 0 --> Cidade 1	R\$ 45	3 ms
Cidade 8 --> Cidade 9	R\$ 45	5 ms
Cidade 14 --> Cidade 17	R\$ 45	6 ms
Cidade 23 --> Cidade 24	R\$ 45	6 ms
Cidade 5 --> Cidade 8	R\$ 50	7 ms
Cidade 10 --> Cidade 13	R\$ 50	6 ms
Cidade 20 --> Cidade 23	R\$ 50	7 ms
Cidade 27 --> Cidade 29	R\$ 50	6 ms
Cidade 16 --> Cidade 19	R\$ 55	8 ms
Cidade 23 --> Cidade 26	R\$ 55	7 ms
Cidade 8 --> Cidade 11	R\$ 60	8 ms

ETAPA 2 - OPERAÇÃO (Algoritmo de Dijkstra)

Latência acumulada da Cidade 0 para todas as outras (usando a rede COMPLETA de conexões disponíveis)

Destino	Latência Acumulada
Cidade 1	3 ms
Cidade 2	5 ms
Cidade 3	9 ms
Cidade 4	9 ms
Cidade 5	10 ms
Cidade 6	13 ms
Cidade 7	18 ms
Cidade 8	16 ms
Cidade 9	18 ms
Cidade 10	25 ms
Cidade 11	24 ms
Cidade 12	27 ms
Cidade 13	30 ms
Cidade 14	31 ms
Cidade 15	33 ms
Cidade 16	39 ms
Cidade 17	37 ms
Cidade 18	41 ms
Cidade 19	47 ms
Cidade 20	46 ms
Cidade 21	49 ms
Cidade 22	51 ms
Cidade 23	52 ms
Cidade 24	57 ms
Cidade 25	59 ms
Cidade 26	58 ms
Cidade 27	62 ms
Cidade 28	68 ms
Cidade 29	68 ms

2. QUESTÃO

Justificativa

Next-Fit

O Next-Fit mantém apenas um servidor aberto por vez e aloca a VM lá se couber, caso contrário abre um novo. É rápido mas tende a desperdiçar espaço pois nunca volta a servidores anteriores.

First-Fit Decreasing

Este produz um melhor resultado ao ordenar as VMs da maior para a menor antes de alocar, as VMs grandes ocupam os servidores primeiro e as pequenas preenchem os espaços que sobram. Isso reduz o desperdício e resulta em menos servidores no total.

Saída

```
[Heurística Next-Fit]
- Servidores utilizados: 24 servidores
- Exemplo de ocupação do Servidor 1: [48, 12, 35] (Total: 95/100 GB)

[Heurística First-Fit Decreasing]
- Servidores utilizados: 20 servidores
- Exemplo de ocupação do Servidor 1: [65, 35] (Total: 100/100 GB)

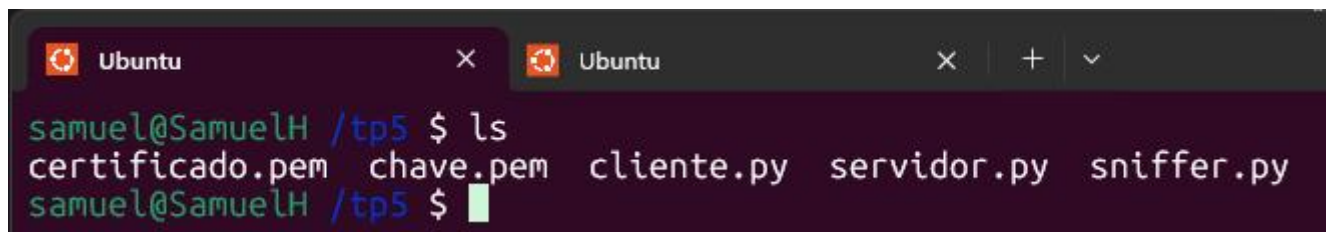
Conclusão: A heurística First-Fit Decreasing economizou 4 servidor(es) em relação à Next-Fit.
```

3. QUESTÃO

Explicação da Saída

O sniffer capturou todos os pacotes trafegados na porta 8443, mas em nenhum momento conseguiu encontrar o padrão AUTH_TOKEN no payload. Isso acontece porque o TLS cifra os dados antes de enviá-los pela rede, então tudo que o sniffer vê são bytes aleatórios e ilegíveis. A mensagem só é decifrada do outro lado, dentro do servidor, após o handshake TLS. Isso prova que a criptografia está funcionando corretamente e impedindo o vazamento de informações.

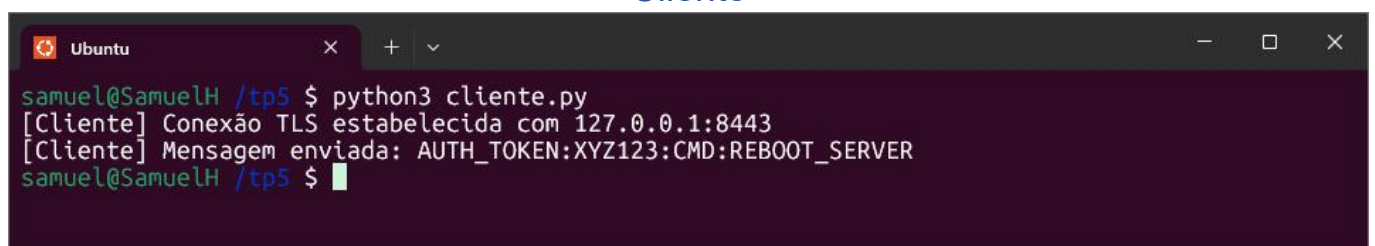
Saída



A terminal window titled 'Ubuntu' showing a user named 'samuel' at host 'SamuelH' in directory '/tp5'. The user has entered the command 'ls', and the output lists four files: 'certificado.pem', 'chave.pem', 'cliente.py', 'servidor.py', and 'sniffer.py'.

```
samuel@SamuelH /tp5 $ ls
certificado.pem  chave.pem  cliente.py  servidor.py  sniffer.py
samuel@SamuelH /tp5 $
```

Cliente



A terminal window titled 'Ubuntu' showing the same user and host. The user has entered the command 'python3 cliente.py'. The output shows two status messages: '[Cliente] Conexão TLS estabelecida com 127.0.0.1:8443' and '[Cliente] Mensagem enviada: AUTH_TOKEN:XYZ123:CMD:REBOOT_SERVER'.

```
samuel@SamuelH /tp5 $ python3 cliente.py
[Cliente] Conexão TLS estabelecida com 127.0.0.1:8443
[Cliente] Mensagem enviada: AUTH_TOKEN:XYZ123:CMD:REBOOT_SERVER
samuel@SamuelH /tp5 $
```

Servidor

```
Ubuntu
samuel@SamuelH /tp5 $ python3 servidor.py
[Servidor] Aguardando conexão em 127.0.0.1:8443...
[Servidor] Cliente conectado: ('127.0.0.1', 38752)
[Servidor] Comando Seguro Recebido: AUTH_TOKEN:XYZ123:CMD:REBOOT_SERVER
samuel@SamuelH /tp5 $
```

Sniffer

```
.R.U..S.RU.
[-] Alerta: Padrão 'AUTH_TOKEN' NÃO encontrado. Os dados estão devidamente cifrados via TLS.

[+] Pacote TCP Capturado! Tamanho: 123 bytes.
[Dados Brutos do Payload]: b'\x01\x01\x08\n\xf6\x82\xf2C\xf6\x82\xf2B\x17\x03\x03\x004\x91GM\xb3\x8c\xa4$a1W\xf7)\xcd#WY\x0c\x9aJyh-\xb6\xaf\x14\xcf\x9e\x01}W\xa3\xc3\xb05U\xf8\xfdB\xfc\xbc~\xb1R\xa7\x84Wy\x8a*\t\xb5\xcd'
[Texto Convertido]: .....C...B....4.GM...$a1W.)#WY...Jyh-.....}W...5U..B...~.R..Wy.*...
[-] Alerta: Padrão 'AUTH_TOKEN' NÃO encontrado. Os dados estão devidamente cifrados via TLS.

[+] Pacote TCP Capturado! Tamanho: 54 bytes.
[-] Pacote sem payload (controle TCP).

[+] Pacote TCP Capturado! Tamanho: 66 bytes.
[Dados Brutos do Payload]: b'\x01\x01\x08\n\xf6\x82\xf2C\xf6\x82\xf2C'
[Texto Convertido]: .....C...C
[-] Alerta: Padrão 'AUTH_TOKEN' NÃO encontrado. Os dados estão devidamente cifrados via TLS.

[+] Pacote TCP Capturado! Tamanho: 54 bytes.
[-] Pacote sem payload (controle TCP).
```

4. QUESTÃO

Lógica do Execução

O programa funciona em duas partes. Primeiro manda pacotes ARP para todos os IPs da rede e espera as respostas, registrando o MAC real de cada dispositivo ativo como "tabela da verdade". Depois fica escutando o tráfego ARP, e se alguém anunciar um MAC diferente do registrado para o IP do gateway, o programa detecta como ataque.

Explicação da Saída

A varredura encontrou só um dispositivo na rede, o gateway 172.18.80.1 com MAC 00:15:5d:99:6f:43. Isso acontece porque o ambiente é WSL, que enxerga poucos dispositivos da rede real. Depois disso o programa entrou em modo de monitoramento e ficou aguardando.

```
Ubuntu
samuel@SamuelH /tp5 $ nano tp5_exercicio04.py
samuel@SamuelH /tp5 $ sudo python3 tp5_exercicio04.py

[*] Iniciando varredura da rede: 172.18.80.0/20
[+] IP: 172.18.80.1          MAC: 00:15:5d:99:6f:43

[*] Varredura concluída. 1 dispositivo(s) encontrado(s).

[*] Monitorando tráfego ARP... (Ctrl+C para parar)
```


5. QUESTÃO

Lógica do que foi feito

O script usa o dnsrecon para coletar registros DNS do zonetransfer.me e o nmap com scripts NSE para varrer portas e identificar vulnerabilidades no scanme.nmap.org.

Explicação da saída

Saída

Foram encontradas duas portas abertas: porta 22 com OpenSSH 6.6.1 e porta 80 com Apache 2.4.7. Os scripts NSE identificaram CVEs críticos em ambos os serviços e uma possível vulnerabilidade Slowloris no servidor web. O relatório foi salvo em relatorio_recon.json.

```
Ubuntu
samuel@SamuelH /tp5 $ sudo python3 tp5_exercicio05.py
[1] Iniciando mapeamento DNS em: zonetransfer.me
Enumerando registros padrão (A, AAAA, MX, TXT, NS)...
[+] ScanInfo -> ? []
[+] SOA -> ? [81.4.108.41]
[+] NS -> ? [5.196.105.10]
[+] NS -> ? [81.4.108.41]
[+] MX -> ? [142.250.102.27]
[+] MX -> ? [192.178.223.27]
[+] MX -> ? [192.178.223.27]
[+] MX -> ? [173.194.76.26]
[+] MX -> ? [173.194.76.27]
[+] MX -> ? [108.177.123.27]
[+] MX -> ? [192.178.213.27]
[+] MX -> ? [2a00:1450:4025:402::1a]
[+] MX -> ? [2a00:1450:4009:c0f::1b]
[+] MX -> ? [2a00:1450:4009:c0f::1a]
[+] MX -> ? [2a00:1450:400c:c00::1b]
[+] MX -> ? [2a00:1450:400c:c00::1a]
[+] MX -> ? [2800:3f0:4003:c00::1b]
[+] MX -> ? [2a00:1450:4013:c1e::1b]
[+] A -> zonetransfer.me [5.196.105.14]
[+] TXT -> zonetransfer.me [google-site-verification=tyP28J7JAUHA9fw2sHXMgcCC0I6XBmmoVi04VlMewxA]
[+] SRV -> _sip._tcp.zonetransfer.me [5.196.105.14]

[2] Iniciando varredura de portas em: scanme.nmap.org
Varrendo top 100 portas com detecção de versão e scripts NSE...

Host: 45.33.32.156
Porta: 22/tcp -> open | OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13
[NSE vulners]:
cpe:/a:openbsd:openssh:6.6.1p1:
DF059135-2CF5-5441-8F22-E6EF1DEE5F6E 10.0 https://vulners.com/gitee/DF059135-2CF5-5441-8F22-E6
EF1DEE5F6E *EXPLOIT*
PACKETSTORM:173661 9.8 https://vulners.
Porta: 25/tcp -> filtered |
Porta: 80/tcp -> open | Apache httpd 2.4.7
[NSE http-stored-xss]: Couldn't find any stored XSS vulnerabilities.
[NSE http-dombased-xss]: Couldn't find any DOM based XSS.
[NSE http-server-header]: Apache/2.4.7 (Ubuntu)
[NSE http-csrf]:
Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=scanme.nmap.org
Found the following possible CSRF vulnerabilities:

Path: http://scanme.nmap.org:80/
Form id: nst-head-sea
[NSE http-enum]:
/images/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'

[NSE vulners]:
cpe:/a:apache:http_server:2.4.7:
```



```

[NSE vulners]:
cpe:/a:apache:http_server:2.4.7:
3E6BA608-776F-5B1F-9BA5-589CD2A5A351 10.0 https://vulners.com/gitee/3E6BA608-776F-5B1F-9BA5-58
9CD2A5A351 *EXPLOIT*
PACKETSTORM:176334 9.8 https://vulners

=====
RELATÓRIO AUTOMATIZADO DE SUPERFÍCIE DE ATAQUE
=====

[+] Alvo DNS analisado : zonetransfer.me
[+] Alvo Nmap analisado: scanme.nmap.org

[1] RESULTADOS DNS (dnsrecon):
- ScanInfo: ? []
- SOA: ? [81.4.108.41]
- NS: ? [5.196.105.10]
- NS: ? [81.4.108.41]
- MX: ? [142.250.102.27]
- MX: ? [192.178.223.27]
- MX: ? [192.178.223.27]
- MX: ? [173.194.76.26]
- MX: ? [173.194.76.27]
- MX: ? [108.177.123.27]
- MX: ? [192.178.213.27]
- MX: ? [2a00:1450:4025:402::1a]
- MX: ? [2a00:1450:4009:c0f::1b]
- MX: ? [2a00:1450:4009:c0f::1a]
- MX: ? [2a00:1450:400c:c00::1b]
- MX: ? [2a00:1450:400c:c00::1a]
- MX: ? [2800:3f0:4003:c00::1b]
- MX: ? [2a00:1450:4013:c1e::1b]
- A: zonetransfer.me [5.196.105.14]
- TXT: zonetransfer.me [google-site-verification=tyP28J7JAUHA9fw2sHXMgcCC0I6XBmmoVi04VLmewxA]
- SRV: _sip._tcp.zonetransfer.me [5.196.105.14]

[2] RESULTADOS DA VARREDURA (Nmap + NSE):
Host: 45.33.32.156
Porta: 22/TCP -> open | ssh OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13
[NSE vulners]:
cpe:/a:openbsd:openssh:6.6.1p1:
DF059135-2CF5-5441-8F22-E6EF1DEE5F6E 10.0 https://vulners.com/gitee/DF059135-2CF5-5441-8F22-E6
EF1DEE5F6E *EXPLOIT*
PACKETSTORM:173661 9.8 https://vulners.
Porta: 25/TCP -> filtered | smtp
Porta: 80/TCP -> open | http Apache httpd 2.4.7
[NSE http-stored-xss]: Couldn't find any stored XSS vulnerabilities.
[NSE http-dombased-xss]: Couldn't find any DOM based XSS.
[NSE http-server-header]: Apache/2.4.7 (Ubuntu)
[NSE http-csrf]:

```

```

[NSE http-dombased-xss]: Couldn't find any DOM based XSS.
[NSE http-server-header]: Apache/2.4.7 (Ubuntu)
[NSE http-csrf]:
Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=scanme.nmap.org
Found the following possible CSRF vulnerabilities:

Path: http://scanme.nmap.org:80/
Form id: nst-head-sea
[NSE http-enum]:
/images/: Potentially interesting directory w/ listing on 'apache/2.4.7 (ubuntu)'

[NSE vulners]:
cpe:/a:apache:http_server:2.4.7:
3E6BA608-776F-5B1F-9BA5-589CD2A5A351 10.0 https://vulners.com/gitee/3E6BA608-776F-5B1F-
9BA5-589CD2A5A351 *EXPLOIT*
PACKETSTORM:176334 9.8 https://vulners

[+] Relatório salvo em: relatorio_recon.json
=====
samuel@SamuelH /tp5 $ ls
questao3 relatorio_recon.json tp5_exercicio04.py tp5_exercicio05.py
samuel@SamuelH /tp5 $ █

```